

Detailed Notes on Introduction to Artificial Intelligence with Python

These notes break down the concepts from the [Introduction to Artificial Intelligence with Python](#) lecture, designed to be accessible for beginners while providing a pathway to advanced understanding. Each section includes explanations, examples, and connections to advanced topics to help you grow your knowledge.



1. Course Overview

What is Artificial Intelligence (AI)?

AI refers to techniques that allow computers to perform tasks that seem "intelligent," mimicking human-like capabilities. Examples include:

- [Recognizing faces](#) in photos (e.g., unlocking your phone with facial recognition).
- [Playing games](#) better than humans (e.g., AlphaGo defeating world champions in Go).
- [Understanding language](#) in voice assistants (e.g., Siri or Alexa responding to commands).

Beginner Insight: Think of AI as teaching a computer to "think" and "act" like a human for specific tasks. For example, when you ask Google Maps for directions, it uses AI to find the best route.

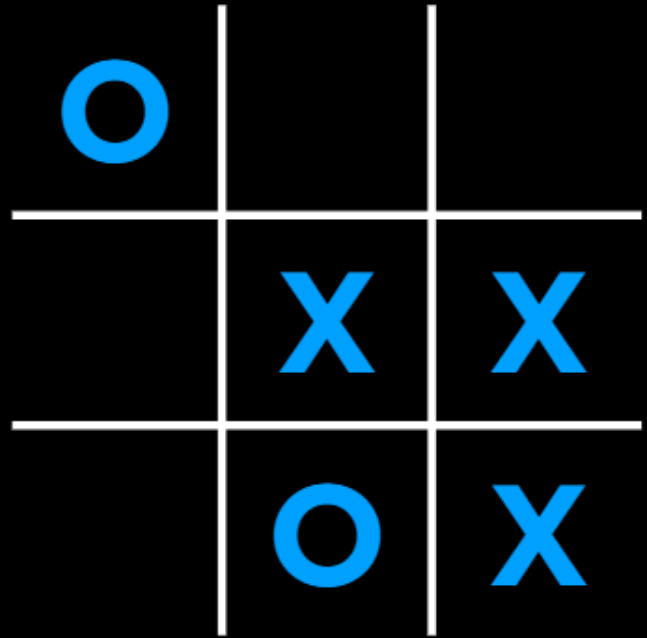
Advanced Connection: AI is a broad field encompassing machine learning, deep learning, and more. These techniques rely on mathematical models, algorithms, and data to simulate intelligent behavior.

Key Topics Covered in the Course

This course introduces the following core areas of AI:

1. **Search**: Finding solutions to problems (e.g., the shortest path in a maze).

Search



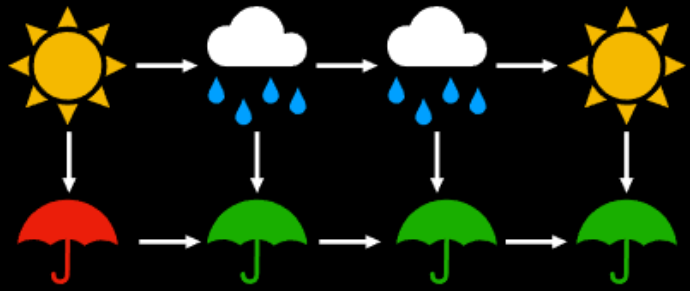
2. **Knowledge**: Representing and reasoning with facts (e.g., diagnosing diseases based on symptoms).

Knowledge

$$\frac{P \rightarrow Q}{P} Q$$

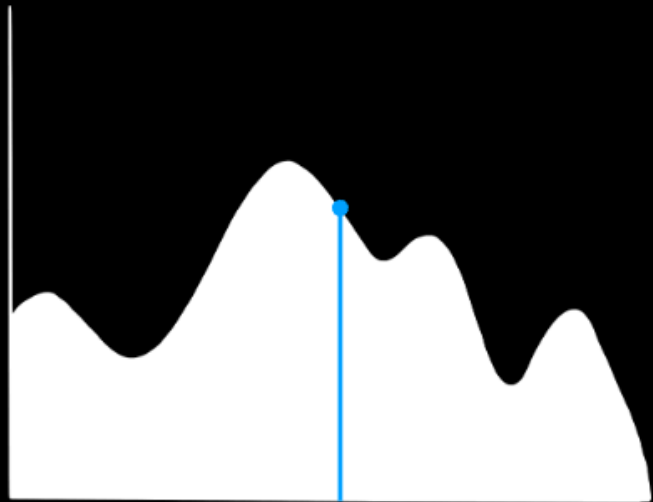
3. **Uncertainty**: Making decisions when outcomes are not guaranteed (e.g., weather forecasting).

Uncertainty

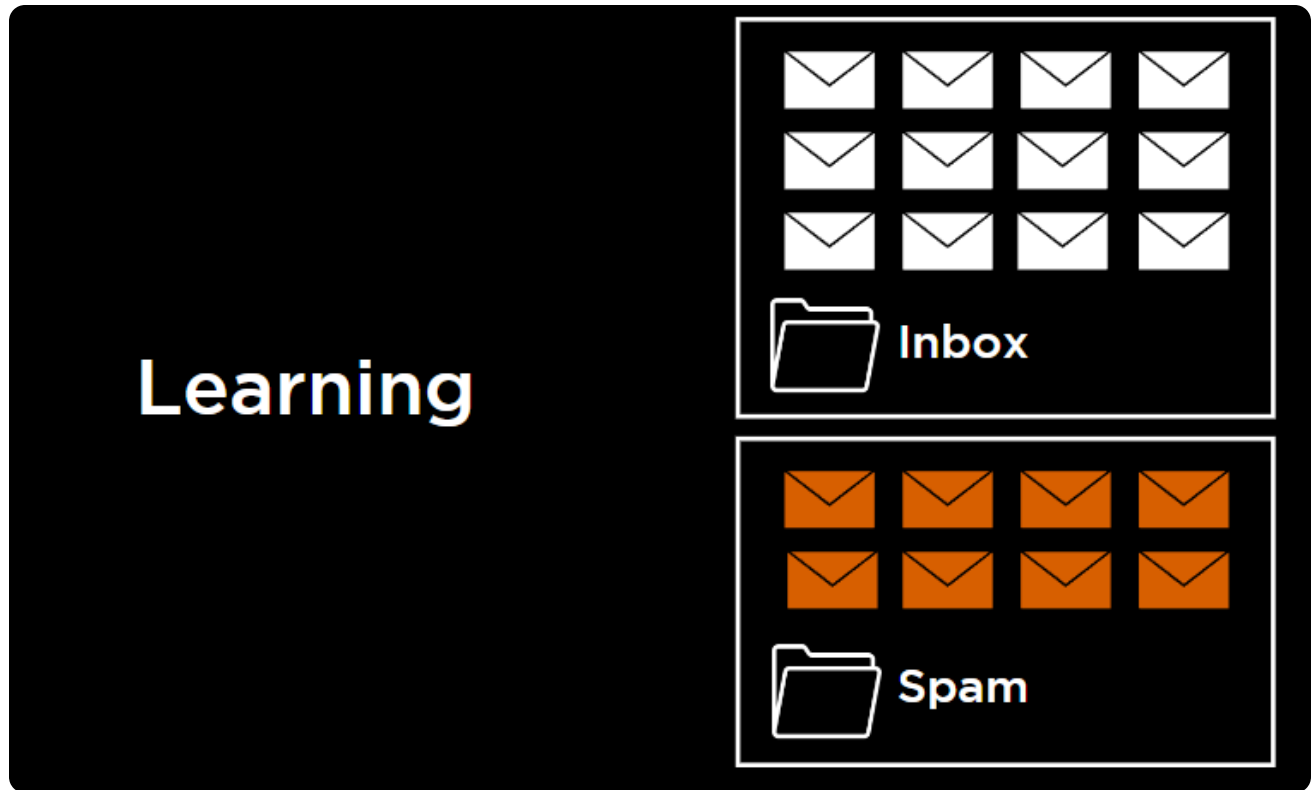


4. **Optimization**: Finding the most efficient solution (e.g., minimizing travel time).

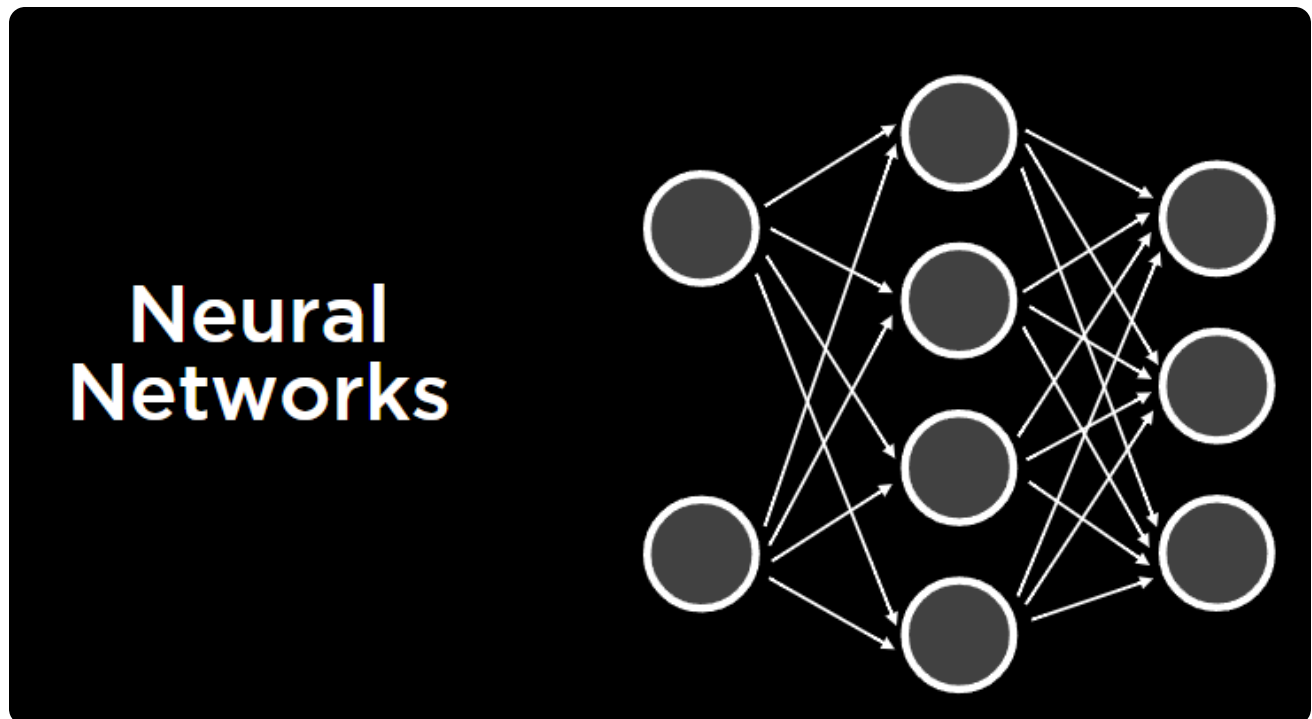
Optimization



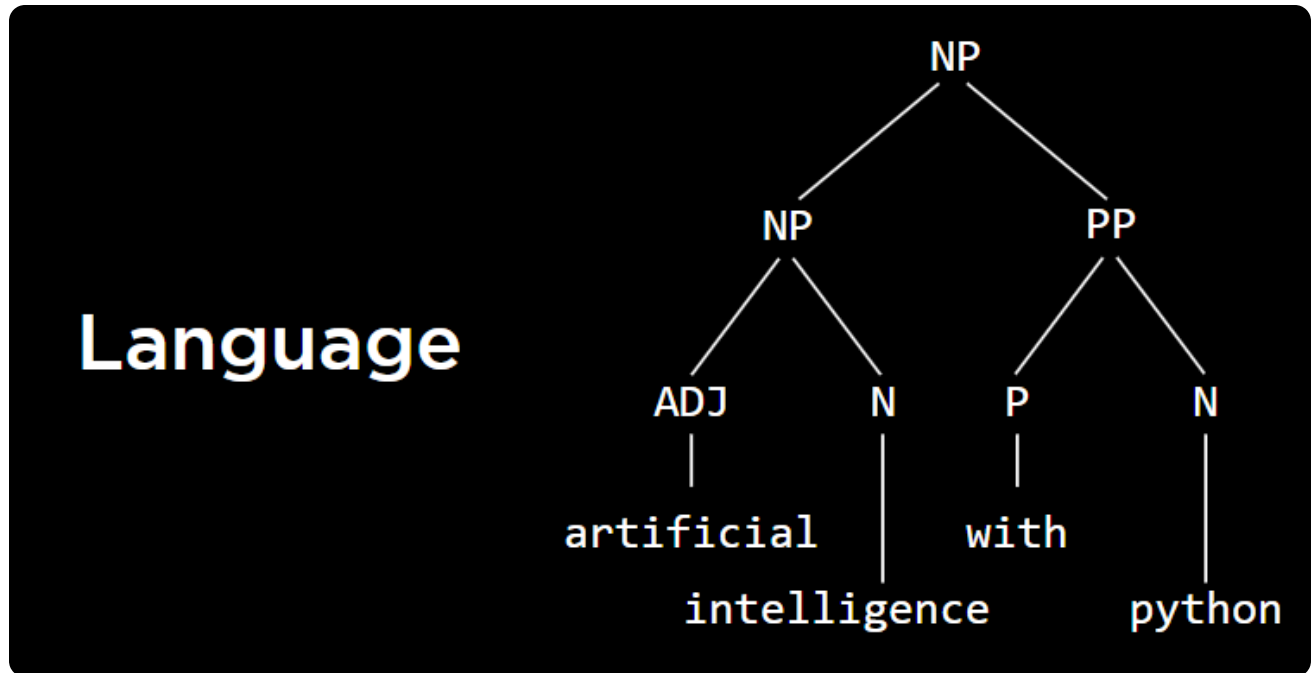
5. **Machine Learning**: Teaching computers to learn from data (e.g., spam email detection).



6. **Neural Networks**: Models inspired by the human brain (e.g., image recognition).



7. **Natural Language Processing (NLP)**: Understanding human language (e.g., chatbots).



Beginner Insight: Each topic is a building block of AI. For now, think of them as tools in a toolbox—each tool solves a different kind of problem.

Advanced Connection: These topics are interconnected. For example, search algorithms can be enhanced with machine learning to make them more efficient, and neural networks often power NLP systems.

Focus of Today's Lecture: Search

Today, we start with **search**, a fundamental concept where an AI finds a sequence of actions to achieve a goal, like navigating a maze or solving a puzzle.

Beginner Insight: Imagine trying to find your way out of a maze. You take steps, explore paths, and backtrack if you hit a dead end. AI search works similarly, but the computer does it systematically.



2. Understanding Search Problems

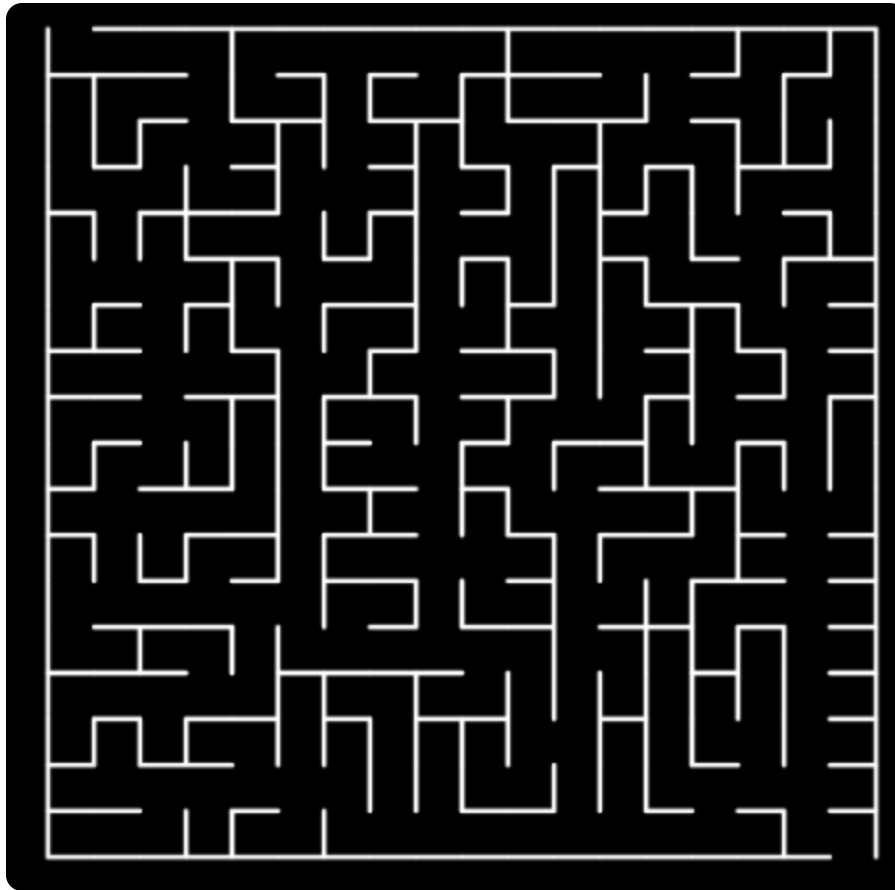
What is a Search Problem?

A search problem is about finding a sequence of actions that takes an AI from a **starting point** (initial state) to a **desired outcome** (goal state). Common examples include:

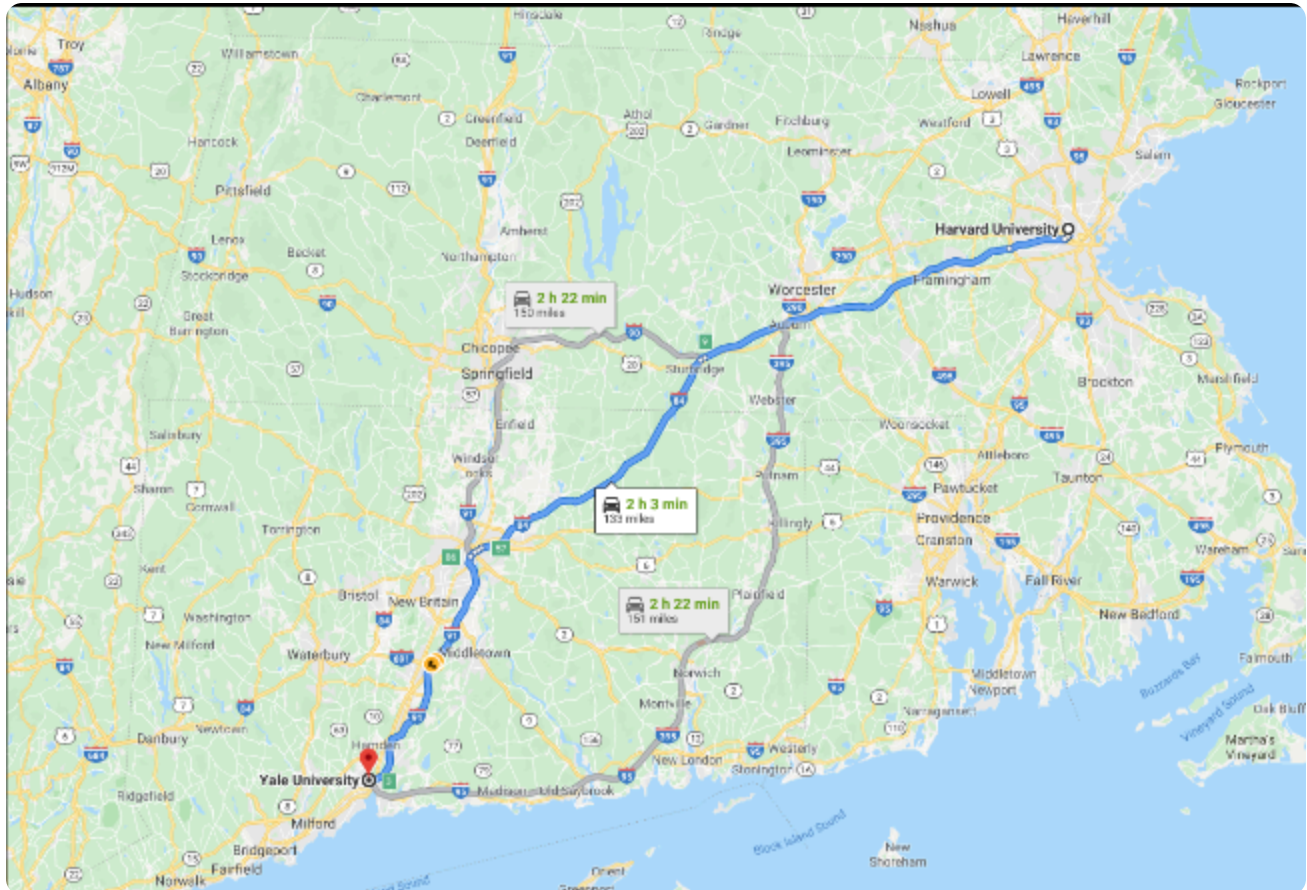
- **15-puzzle**: Sliding tiles to arrange numbers in order (e.g., 1 to 15 in a 4x4 grid).



- **Maze navigation:** Finding a path from the entrance to the exit.



- **Driving directions:** Calculating the fastest route from point A to point B.



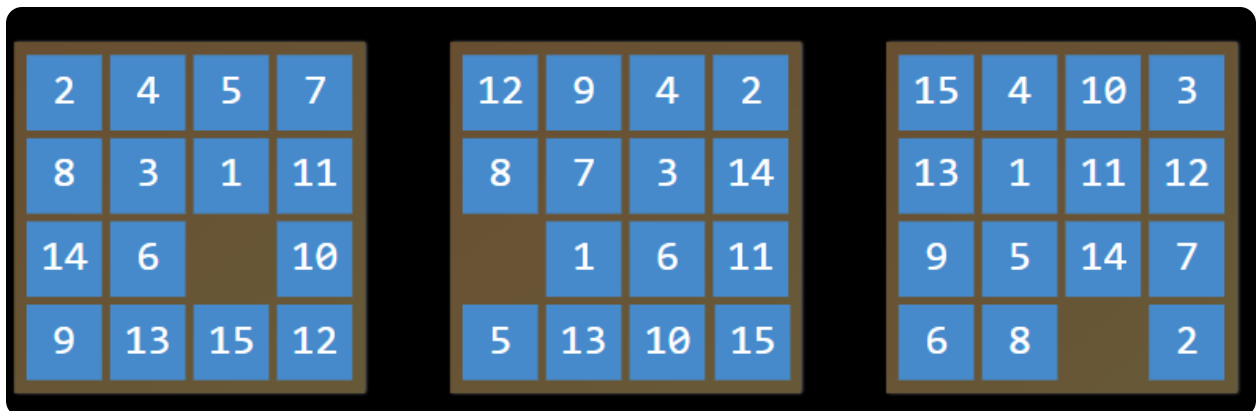
Beginner Insight: Think of a search problem like planning a trip. You start at home (initial state), choose roads to take (actions), and aim to reach your destination (goal state).

Advanced Connection: Search problems are formalized in AI using graphs, where nodes represent states and edges represent actions. This allows algorithms to systematically explore possibilities.

Key Components of a Search Problem

To solve a search problem, we define it using specific terms:

1. **Agent:** The entity making decisions (e.g., a virtual car in a navigation app or the AI solving a puzzle).
 - **Example:** In a maze, the agent is the "character" moving through it.
2. **State:** A snapshot of the situation at a given moment (e.g., the current arrangement of tiles in a 15-puzzle).
 - **Example:** In a 15-puzzle, one state might have the tiles in a specific order, like [1, 2, 3, 4, 5, ..., 15, blank].



3. **Initial State:** Where the agent starts.
 - **Example:** The starting arrangement of tiles in a 15-puzzle.

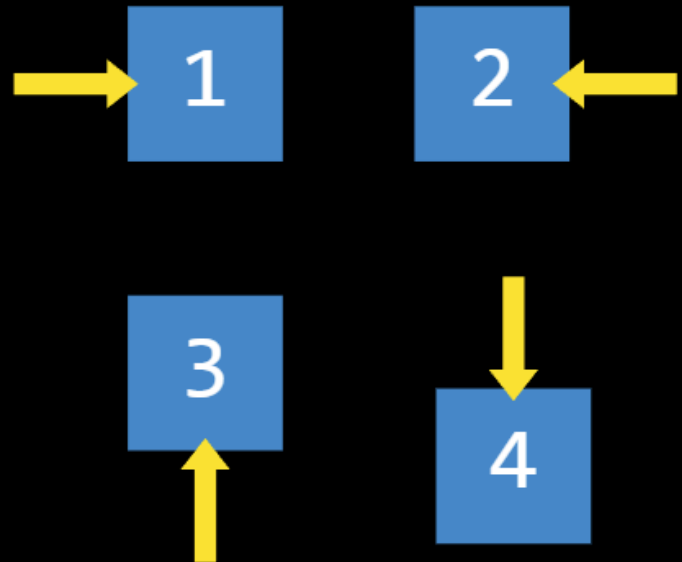
initial state

2	4	5	7
8	3	1	11
14	6		10
9	13	15	12

4. **Actions:** Choices the agent can make in a given state.

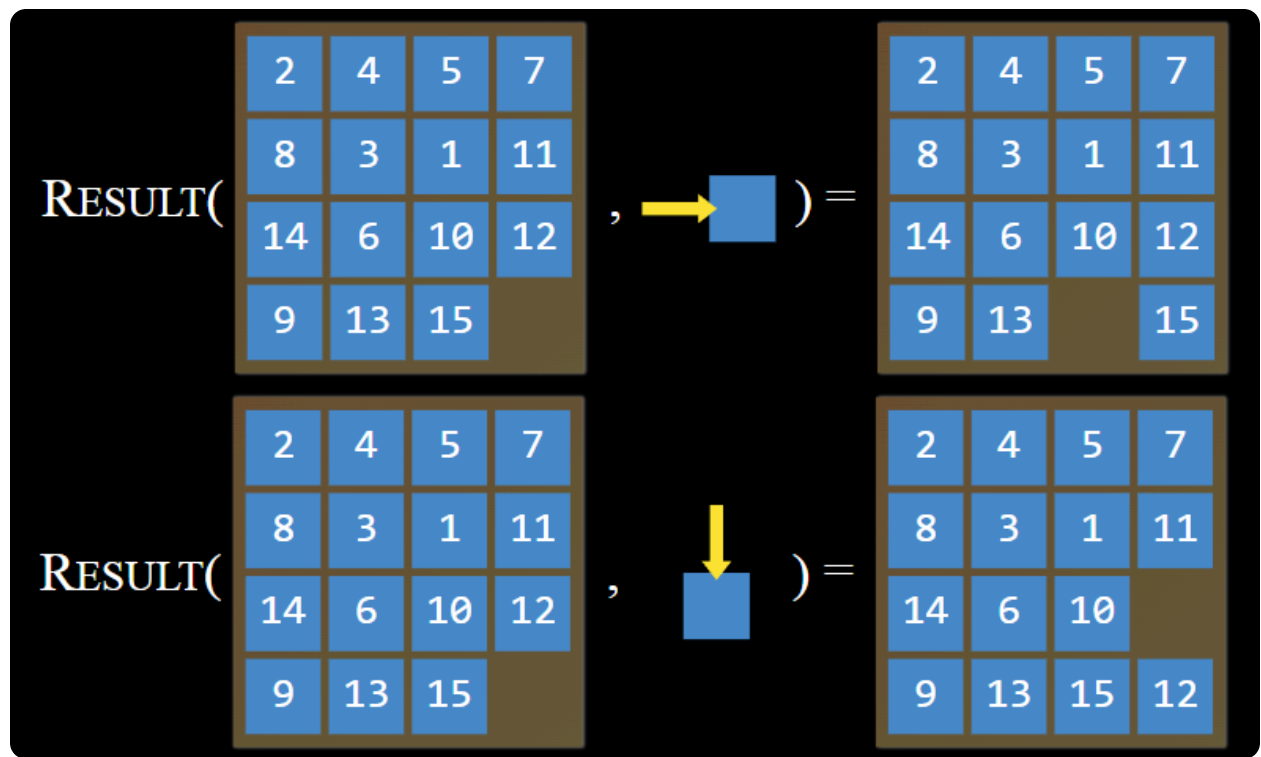
- **Formal Definition:** A function `actions(s)` takes a state `s` and returns all possible actions.
- **Example:** In a 15-puzzle, actions might be "slide the blank tile up," "down," "left," or "right."

actions



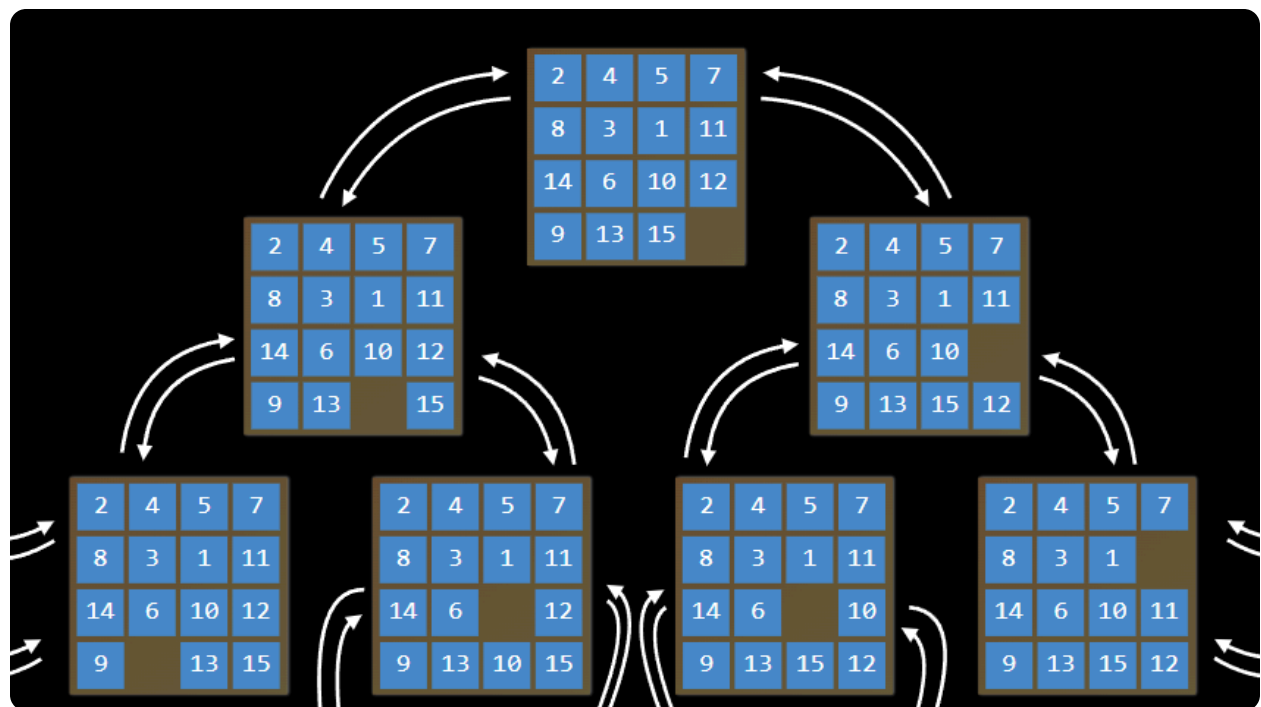
5. **Transition Model:** Describes what happens when an action is taken.

- **Formal Definition:** A function `result(s, a)` takes a state `s` and action `a`, returning the new state.
- **Example:** If you slide a tile right in a 15-puzzle, the transition model gives you the new tile arrangement.



6. **State Space:** All possible states the agent can reach by applying actions, visualized as a graph where:

- **Nodes:** States (e.g., every possible tile arrangement).
- **Edges:** Actions connecting states (e.g., sliding a tile).
- **Example:** In a maze, the state space is all possible positions you can reach.

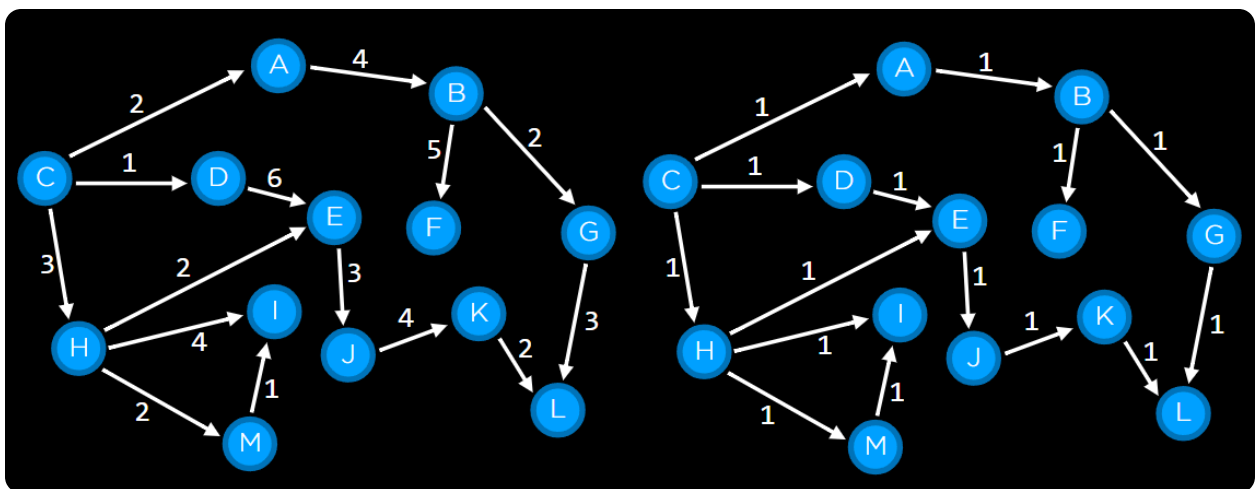
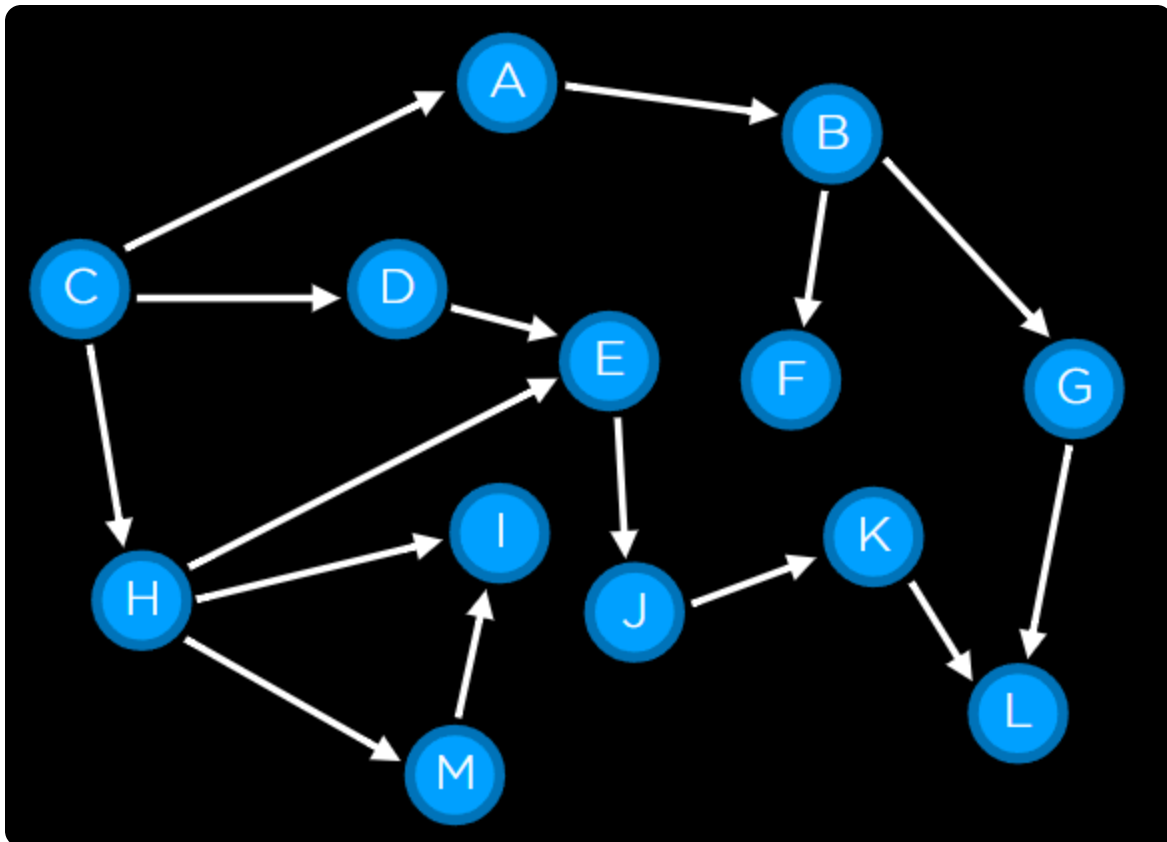


7. **Goal Test:** A function to check if a state is the desired goal.

- **Example:** For a 15-puzzle, the goal test checks if tiles are in order (1, 2, 3, ..., 15, blank).

8. **Path Cost:** A number representing the cost of a sequence of actions (e.g., time, distance, or number of moves).

- **Example:** In driving directions, the path cost might be the total distance traveled. The goal is often to minimize this cost.



Beginner Insight: Imagine playing a board game. The board's current setup is the **state**, your possible moves are **actions**, and the rules of the game act like the **transition model**. Your goal (e.g., winning) is checked by the **goal test**, and you might want to win in the fewest moves (**path cost**).

Advanced Connection: These components form a mathematical framework called a **state-space graph**. Advanced AI techniques, like heuristic search, optimize how we explore this

graph to find solutions faster.

What Is A Search Problems

A **search problem** is a formal framework in AI and computer science to model and solve problems where an agent finds a sequence of actions from an initial state to a goal state.

1. Initial State

- **Definition:** The starting configuration of the problem from which the search begins.
- **Role:** Defines the agent's starting position or situation.
- **Key Points:**
 - Unique and fully specified.
 - Serves as the root of the search tree or graph.

2. Actions

- **Definition:** The set of possible operations or moves the agent can perform in a state.
- **Role:** Determines how the agent transitions between states.
- **Key Points:**
 - Often state-dependent (e.g., limited by constraints like boundaries).
 - Typically a finite set, varying by state.

3. Transition Model

- **Definition:** A function describing the new state resulting from taking an action in a given state.
- **Role:** Specifies the rules governing how actions change the system's state.
- **Key Points:**
 - Denoted as `Result(state, action) → new_state`.
 - Can be deterministic or stochastic.

4. Goal Test

- **Definition:** A condition or function determining if a state is a goal state (solution).
- **Role:** Indicates when the search has found a solution, stopping exploration.
- **Key Points:**
 - Boolean function: `IsGoal(state) → true/false`.
 - May have one or multiple goal states.

5. Path Cost Function

- **Definition:** A function assigning a numerical cost to a sequence of actions (path).
- **Role:** Quantifies the expense of a path to prioritize cheaper solutions.
- **Key Points:**
 - Denoted as `g(path)` or `g(state)`, summing action costs.

- Costs are non-negative; goal is often to minimize total cost.

In short a search problem is defined by:

- **Initial State**: Starting point.
- **Actions**: Possible moves.
- **Transition Model**: Rules for state changes.
- **Goal Test**: Solution check.
- **Path Cost Function**: Path expense.

These components form a state space (graph or tree) where search algorithms explore paths to find a goal state with minimal cost.

Goal of Search

The aim is to find a **solution**—a sequence of actions that takes the agent from the initial state to a goal state.

The **optimal solution** is the one with the lowest path cost.

Beginner Insight: Think of finding the shortest route on Google Maps. The solution is the list of turns to take, and the optimal solution is the fastest or shortest route.

Advanced Connection: Finding the optimal solution often involves algorithms like A* search, which uses heuristics to prioritize exploring promising paths.

»

3. Search Algorithm Framework

How Search Algorithms Work

To solve a search problem, we use a **node**

Node

A Data Structure That Keeps Track Of:-

- **State**: The current configuration (e.g., a specific tile arrangement).
- **Parent**: The previous node that led to this state (helps backtrack to find the solution).
- **Action**: The action taken to reach this state (e.g., "slide tile right").
- **Path Cost**: The total cost from the initial state to this state (e.g., number of moves).

The algorithm uses a **frontier**, a collection of nodes waiting to be explored. The basic steps are:

1. Start with the frontier containing only the **initial state** node.
2. Repeat until a solution is found:
 - If the frontier is empty, there's no solution (failure).
 - Pick a node from the frontier.
 - Check if its state is a goal state (using the goal test). If yes, backtrack through parent nodes to get the sequence of actions (the solution).
 - If not, **expand** the node:
 - Use the transition model to generate all possible next states.
 - Create new nodes for these states and add them to the frontier.

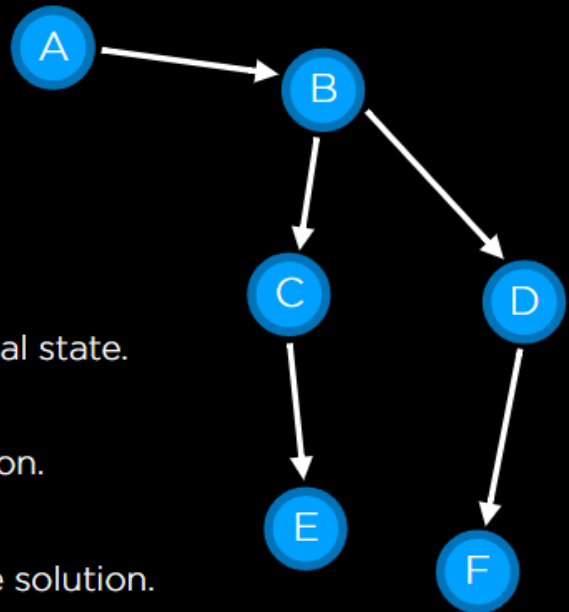
Find a path from A To E

Consider a graph where the goal is to find a path from node A to node E:

Find a path from A to E.

Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



- **Initial Frontier:** {A}

Frontier

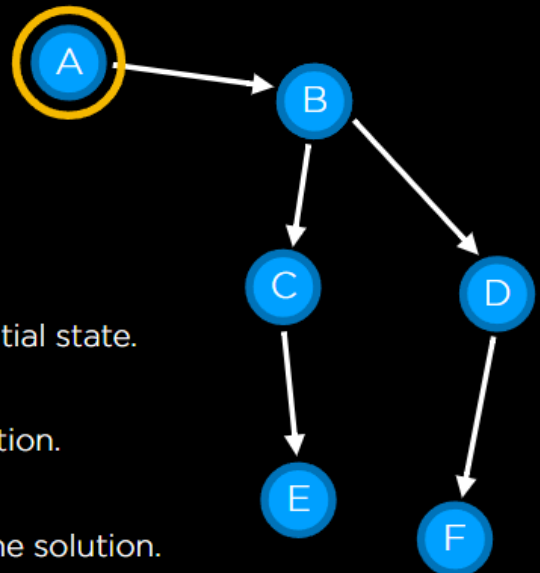


- **Step 1:** Remove A, not the goal. Expand A → add B to frontier.

Find a path from A to E.

Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



- **Frontier:** {B}

Frontier

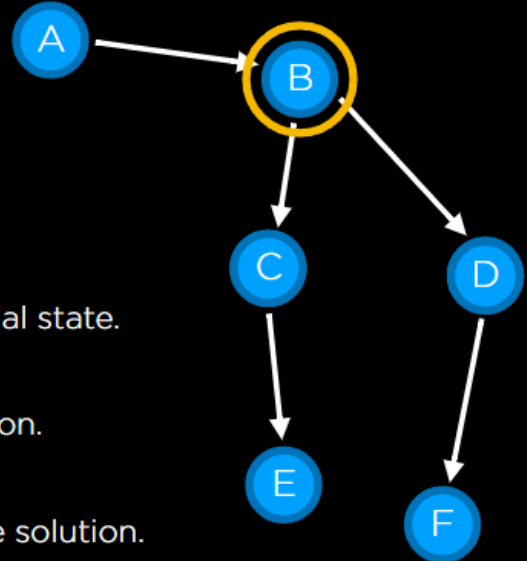


- **Step 2:** Remove B, not the goal. Expand B → add C, D to frontier.

Find a path from A to E.

Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

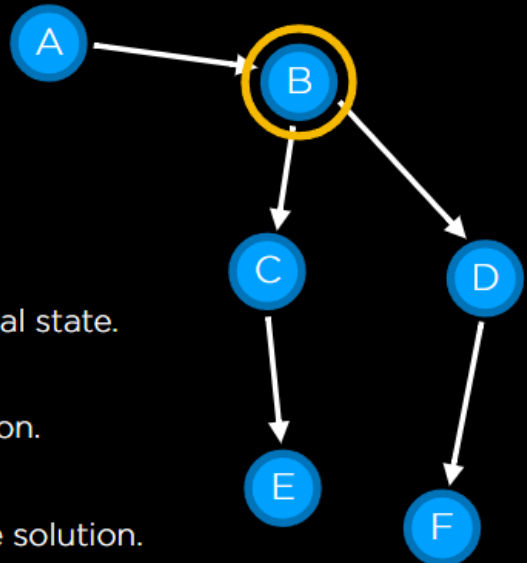


Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



- **Frontier:** {C, D}

Frontier



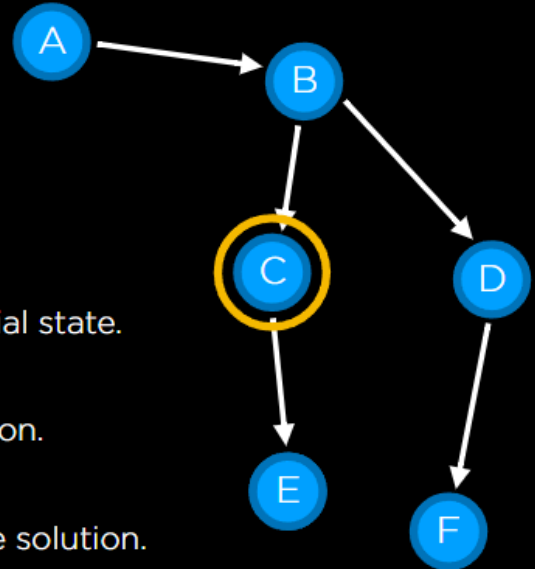
- **Step 3:** Remove C, not the goal. Expand C → add E to frontier.

Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

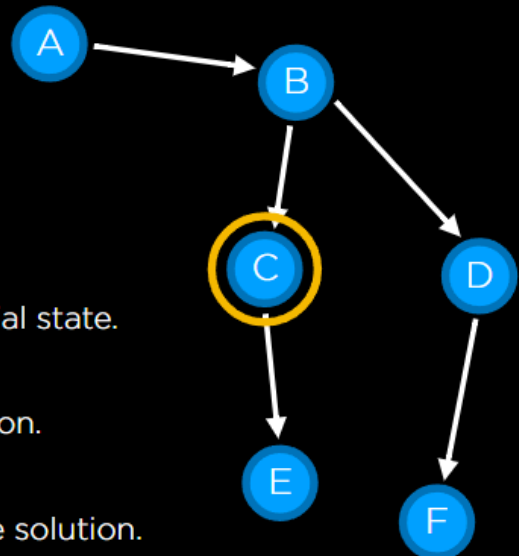


Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



- **Frontier:** {D, E}

Frontier



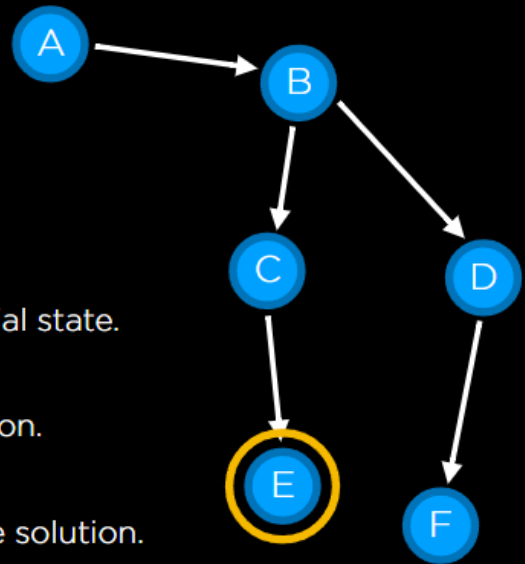
- **Step 4:** Remove E, goal found!

Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



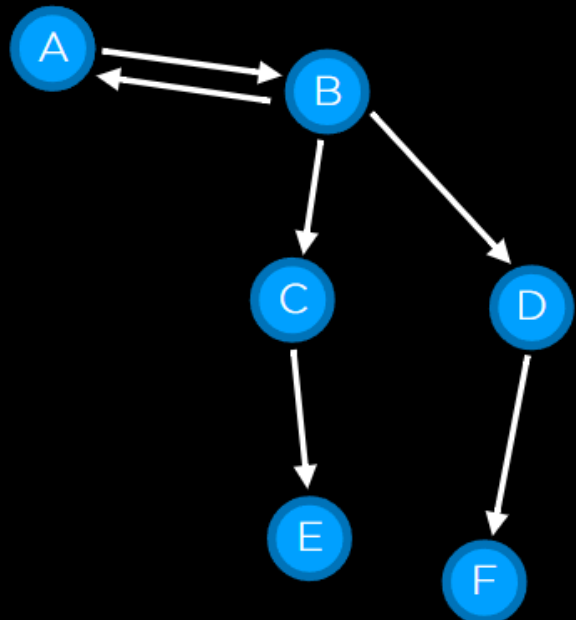
- **Backtrack to get path:** $A \rightarrow B \rightarrow C \rightarrow E$

What Can Go Wrong ?

In problems like the 15-puzzle, actions can be undone (e.g., slide a tile right, then left), which can cause the algorithm to loop forever. To prevent this:

Find a path from A to E.

Frontier



Revised Approach

- Maintain an **explored set**, a list of states the algorithm has already visited.
- Revised Approach:
 1. Start with the frontier containing the initial state and an empty explored set.
 2. Repeat:
 - If the frontier is empty, report no solution.
 - Pick/Remove a node from the frontier.
 - If node contains a goal state, return the solution.
 - Add the node's state to the explored set.
 - Expand the node, but only add new nodes to the frontier if their states are **not** in the frontier or explored set.

Think of the frontier as a "to-do list" of places to explore in a maze. You pick one spot, check if it's the exit, and if not, add nearby spots to your list.

The frontier's structure determines the search strategy. For example:

- A **stack** (last-in, first-out) leads to depth-first search.
- A **queue** (first-in, first-out) leads to breadth-first search.
- A **priority queue** (based on cost or heuristics) leads to algorithms like A*.

Beginner Insight: The explored set is like marking visited rooms in a maze so you don't keep going back to the same ones.

Advanced Connection: The explored set ensures efficiency by preventing redundant exploration. In advanced algorithms, techniques like pruning or heuristic functions further optimize this process.



Depth-First Search (DFS)

A **stack** (last-in, first-out) leads to depth-first search.

Last Node To Go In Will Be The First To Thrown Out

Frontier

- **Definition:** The frontier is the set of nodes (states) that are yet to be explored in the search process.
- **Implementation:** Implemented as a **stack** (Last-In, First-Out, LIFO) data structure.
 - Nodes are added (pushed) to the top of the stack, and the most recently added node is removed (popped) first for exploration.
 - This can be implemented explicitly using a stack data structure or implicitly through recursion, where the call stack manages the LIFO behavior.
- **Why a Stack?:** The stack ensures that DFS prioritizes exploring the **deepest** node (most recently discovered) in the current path, leading to a "dive deep" approach.

Behavior

- **Exploration Strategy:** DFS always selects the **most recently added node** (the deepest node in the current path) from the frontier for exploration.
 - It continues to explore deeper along a single path until it either:
 - Reaches the **goal state** (solution found).
 - Hits a **dead end** (no further actions/children available), at which point it **backtracks** to the most recent node with unexplored children.
- **Backtracking:** When a dead end is reached, DFS retraces its steps to the last node with unvisited neighbors and explores a new path from there.
- **Intuitive Analogy:** Imagine exploring a maze by walking as far as possible down one path until you hit a wall, then turning back to the last junction and trying a different route.
- **Key Characteristic:** DFS prioritizes **depth** over breadth, exploring one branch of the search tree fully before considering others.

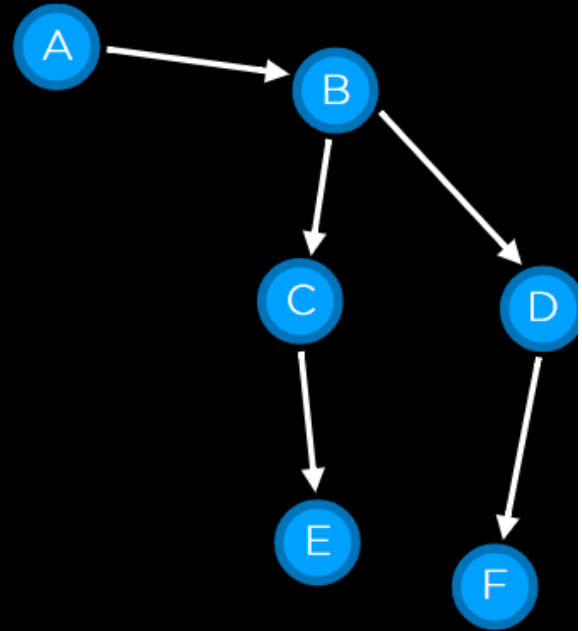
Example on a Graph (A → E)

Consider a directed graph with nodes {A, B, C, D, E, F} and edges as shown in the diagram below. The goal is to find a path from node **A** to node **E**.

Find a path from A to E.

Frontier

Explored Set



1. **Initialization:**

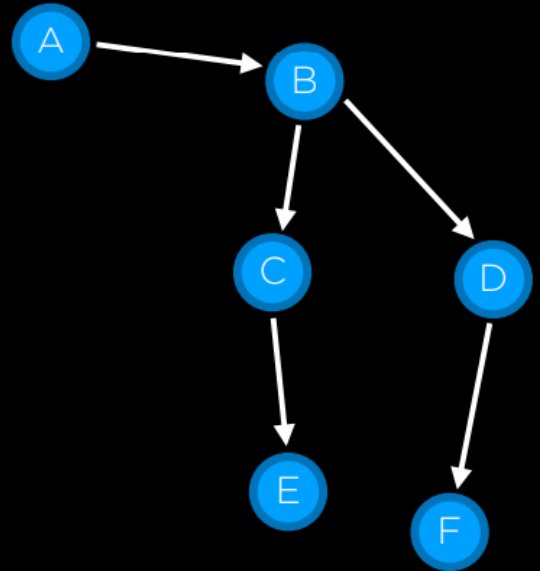
- Frontier = {A} (start node pushed onto the stack).
- Explored = {} (set of visited nodes, initially empty).
- Path = {} (tracks the path to reconstruct the solution).

Find a path from A to E.

Frontier



Explored Set



2. **Step 1:**

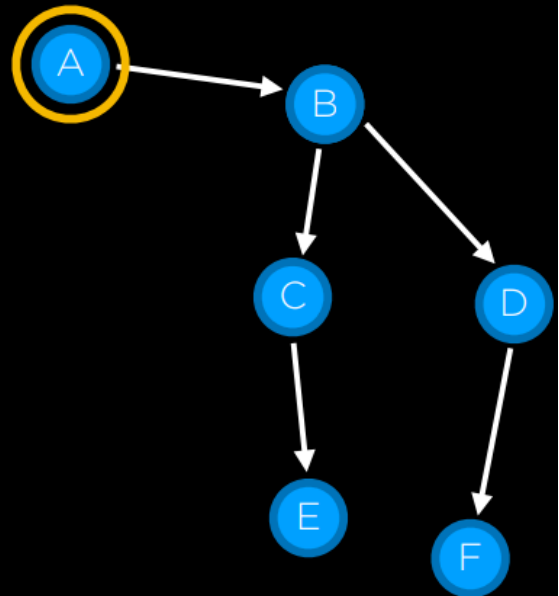
- Pop A from the frontier (Frontier = {}).

Find a path from A to E.

Frontier

Explored Set

A



- A is not the goal (E). Add A to Explored = {A}.
- A has one neighbor: B. Push B onto the stack.
- Frontier = {B}, Explored = {A}, Path = {B → A}.

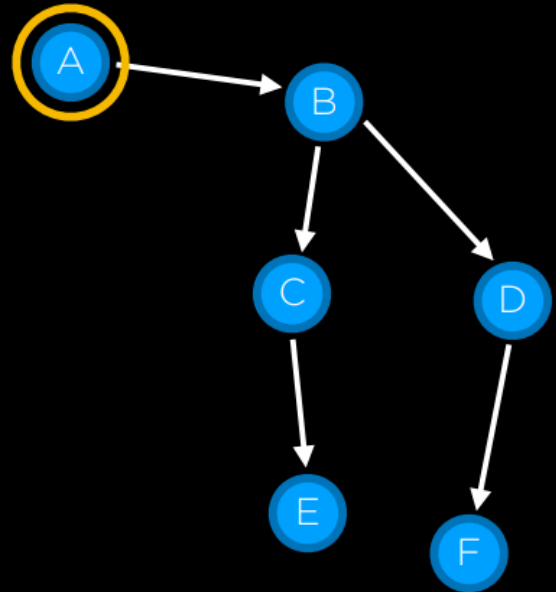
Find a path from A to E.

Frontier

B

Explored Set

A



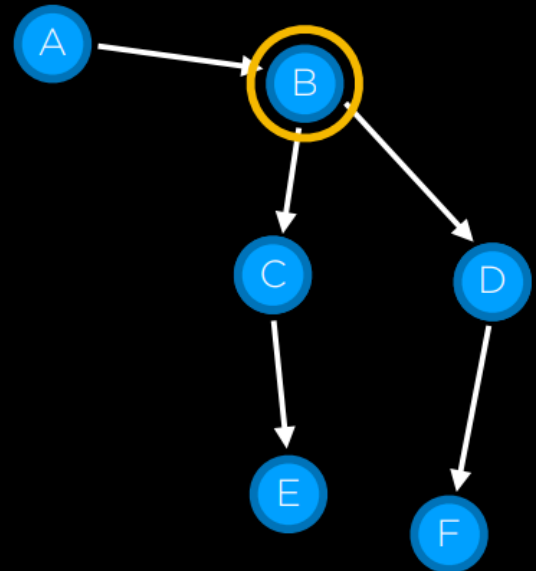
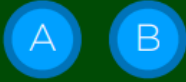
3. **Step 2:**

- Pop B from the frontier (Frontier = {}).

Find a path from A to E.

Frontier

Explored Set



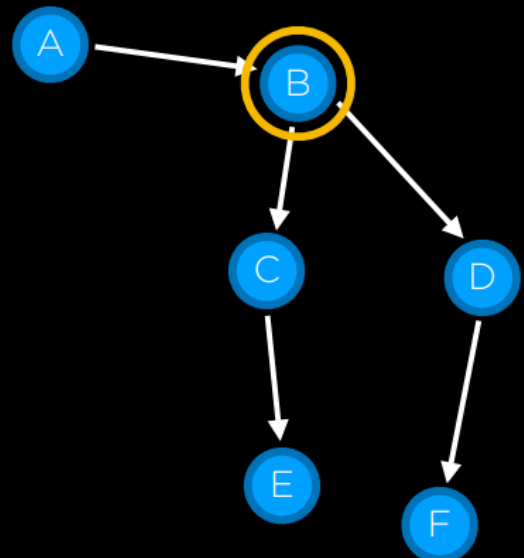
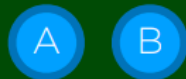
- B is not the goal. Add B to Explored = {A, B}.
- B has two neighbors: D and C. Push them in stack order (C, then D, as stack adds to the top).
- Frontier = {D, C}, Explored = {A, B}, Path = {B → A, D → B, C → B}.

Find a path from A to E.

Frontier



Explored Set



4. **Step 3:**

- Pop D from the frontier (Frontier = {C}).

Find a path from A to E.

Frontier

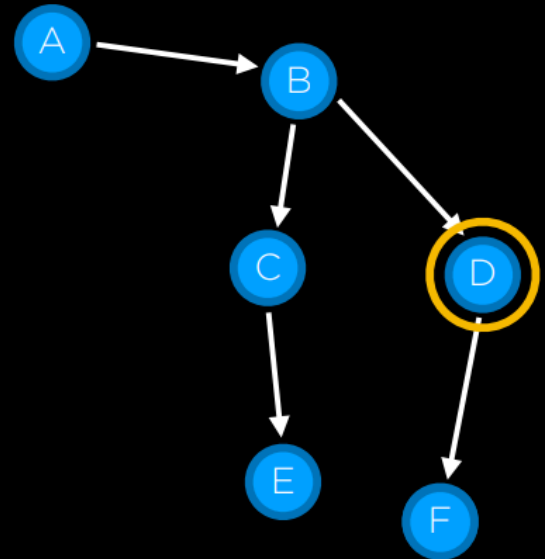
C

Explored Set

A

B

D



- D is not the goal. Add D to Explored = {A, B, D}.
- D has one neighbor: F. Push F onto the stack.
- Frontier = {F, C}, Explored = {A, B, D}, Path = {B → A, D → B, C → B, F → D}.

Find a path from A to E.

Frontier

C

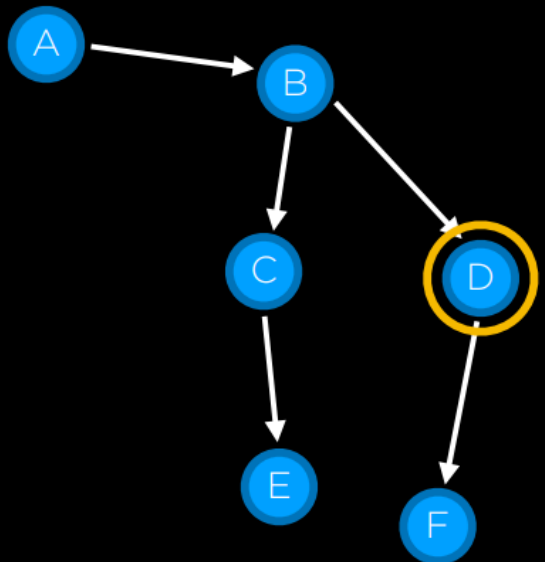
F

Explored Set

A

B

D



5. **Step 4:**

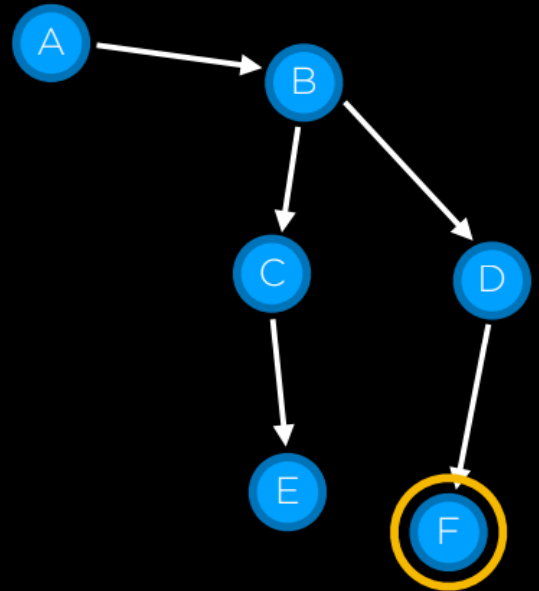
- Pop F from the frontier (Frontier = {C}).

Find a path from A to E.

Frontier



Explored Set



- F is not the goal. Add F to Explored = {A, B, D, F}.
- F has no neighbors (dead end). No nodes to push.
- Frontier = {C}, Explored = {A, B, D, F}, Path unchanged.

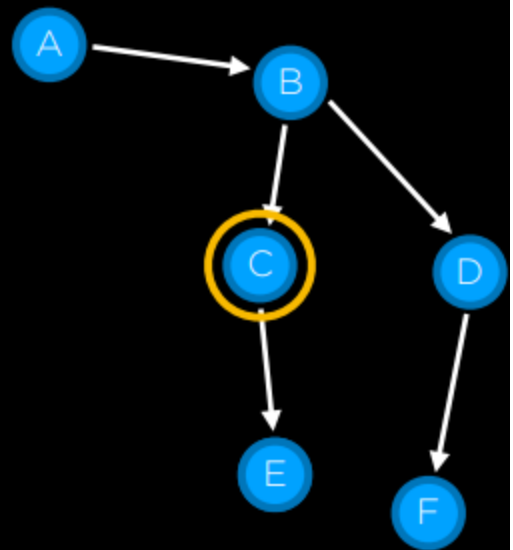
6. **Step 5:**

- Pop C from the frontier (Frontier = {}).

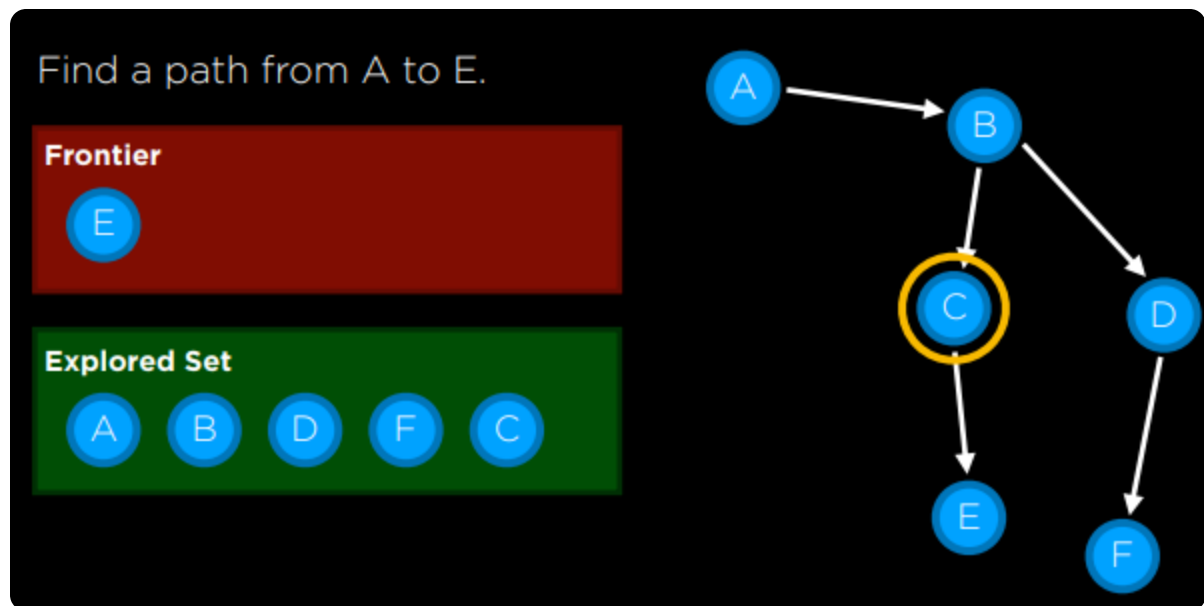
Find a path from A to E.

Frontier

Explored Set

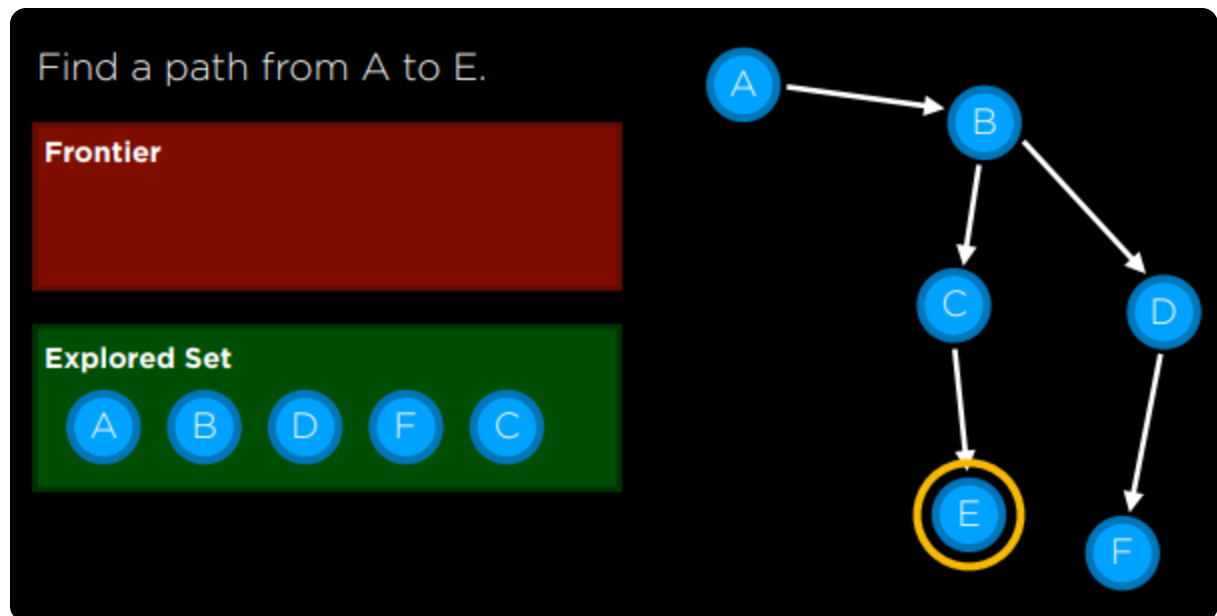


- C is not the goal. Add C to Explored = {A, B, D, F, C}.
- C has one neighbor: E. Push E onto the stack.
- Frontier = {E}, Explored = {A, B, D, F, C}, Path = {B → A, D → B, C → B, F → D, E → C}.



7. **Step 6:**

- Pop E from the frontier (Frontier = {}).
- E is the goal! Stop the search.
- Reconstruct the path using the Path dictionary: $E \rightarrow C \rightarrow B \rightarrow A$ (reverse order: $A \rightarrow B \rightarrow C \rightarrow E$).
- **Solution Path:** $A \rightarrow B \rightarrow C \rightarrow E$.



Note: The path found ($A \rightarrow B \rightarrow C \rightarrow E$) is a valid solution, but DFS does not guarantee the shortest path unless additional mechanisms (e.g., path cost tracking) are added.

Maze Example

- **Scenario:** Imagine navigating a maze where each cell is a state, and edges represent valid moves (e.g., up, down, left, right) to adjacent cells without walls.
- **DFS in a Maze:**
 - Start at the entrance (initial state).
 - Choose one direction and move as far as possible along that path until you hit a dead end (a cell with no unvisited neighbors).
 - Backtrack to the last cell with unexplored neighbors and try a different direction.
 - Repeat until the exit (goal state) is found or all paths are exhausted.
- **Visualization:**
 - DFS might lead you down a long, winding corridor in the maze, only to hit a wall, forcing you to backtrack to an earlier junction.
 - This contrasts with breadth-first search (BFS), which explores all directions level by level, potentially finding the exit sooner if it's closer to the entrance.
- **Example Path:**
 - If the maze has multiple paths to the exit, DFS might take a longer, circuitous route because it commits to one path before exploring alternatives.
 - For instance, DFS might explore a path of 20 steps before backtracking, even if a 5-step path exists.

Properties

1. **Completeness:**
 - DFS is **complete** in **finite state spaces**, meaning it will find a solution if one exists, provided the search avoids infinite loops (e.g., by tracking visited nodes in the Explored set).
 - In **infinite state spaces** or graphs with cycles, DFS may get stuck in an infinite path unless cycle detection is implemented (e.g., checking the Explored set before expanding a node).
 - **Solution:** Always maintain an Explored set to avoid revisiting nodes in graphs with cycles.
2. **Optimality:**
 - DFS is **not optimal**. It does not guarantee the shortest path to the goal, even in weighted graphs or mazes with multiple valid paths.
 - Example: In the maze, DFS might find a 20-step path to the exit when a 5-step path exists, as it explores deeply rather than systematically checking all distances.
 - To achieve optimality, DFS would need modifications (e.g., incorporating path cost comparisons, as in Dijkstra's algorithm or A*).

3. Memory Efficiency:

- DFS is **memory-efficient** compared to breadth-first search (BFS).
- It only stores:
 - The **current path** (nodes from the start to the current node).
 - The **Explored set** (to avoid cycles in graphs).
 - The **frontier** (nodes yet to be explored, typically small since it only grows along one path at a time).
- In contrast, BFS stores all nodes at the current depth level, which can grow exponentially in wide search trees.
- Example: In a binary tree with depth (d), DFS stores at most (d) nodes in the frontier (one per level), while BFS may store (2^d) nodes at the deepest level.

4. Time Complexity:

- In the worst case, DFS explores all nodes in the state space.
- For a graph with (V) nodes and (E) edges, the time complexity is **$O(V + E)$** with an adjacency list representation (assuming cycle detection).
- In a tree with branching factor (b) and depth (d), the worst-case time complexity is $O(b^d)$ if the entire tree is explored.

5. Space Complexity:

- Space complexity is **$O(d)$** for a tree (where (d) is the maximum depth), as the frontier only stores nodes along the current path.
- In a graph, additional space for the Explored set gives a worst-case space complexity of **$O(V)$** (to store all visited nodes).

Additional Notes

• Applications:

- DFS is used in scenarios where memory efficiency is critical, and finding any solution (not necessarily the shortest) is acceptable.
- Examples: Topological sorting, detecting cycles in graphs, solving puzzles (e.g., Sudoku), pathfinding in games (where a solution is needed quickly).

• Variants:

- **Iterative Deepening DFS (IDDFS)**: Combines DFS's memory efficiency with BFS's guarantee of finding the shortest path in unweighted graphs by running DFS with increasing depth limits.
- **Limited-Depth DFS**: Restricts DFS to a maximum depth to avoid infinite loops in infinite state spaces.

• Advantages:

- Simple to implement (especially recursively).
- Low memory usage, making it suitable for large state spaces with deep solutions.
- **Disadvantages:**
 - Can get stuck in deep or infinite paths without cycle detection.
 - Non-optimal solutions may be found, which can be problematic in applications requiring the shortest path.
- **Implementation Tip:**
 - Use a **set** for the Explored set to ensure $O(1)$ lookup when checking for visited nodes.
 - Use a **dictionary** to track the parent of each node (Path) for reconstructing the solution path.

Pseudocode for DFS (Graph Search)

```

function DFS(graph, start, goal):
    frontier = Stack() # Initialize stack for frontier
    frontier.push(start)
    explored = Set() # Track visited nodes
    parent = Dictionary() # Track path (parent[node] = previous node)

    while frontier is not empty:
        current = frontier.pop() # Get the most recent node
        if current == goal:
            return reconstruct_path(parent, goal) # Goal found, return path
        if current not in explored:
            explored.add(current) # Mark as visited
            for neighbor in graph.neighbors(current):
                if neighbor not in explored:
                    frontier.push(neighbor) # Add unvisited neighbors
                    parent[neighbor] = current # Track parent for path

    return None # No solution found

function reconstruct_path(parent, goal):
    path = [goal]
    current = goal
    while current in parent:
        current = parent[current]
        path.append(current)
    return reverse(path) # Return path from start to goal

```

PYTHON

Breadth-First Search (BFS)

Frontier

- **Definition:** The frontier is the set of nodes (states) that are yet to be explored during the search process.
- **Implementation:** Implemented as a **queue** (First-In, First-Out, FIFO) data structure.
 - Nodes are added (enqueued) to the back of the queue, and the oldest node (the one added first) is removed (dequeued) from the front for exploration.
 - This can be implemented using a standard queue (e.g., a list or deque in programming languages like Python) to ensure FIFO behavior.
- **Why a Queue?:** The queue ensures that BFS explores nodes in the order they are discovered, prioritizing nodes at the **current depth level** before moving to nodes at a deeper level.

queue

first-in first-out data type

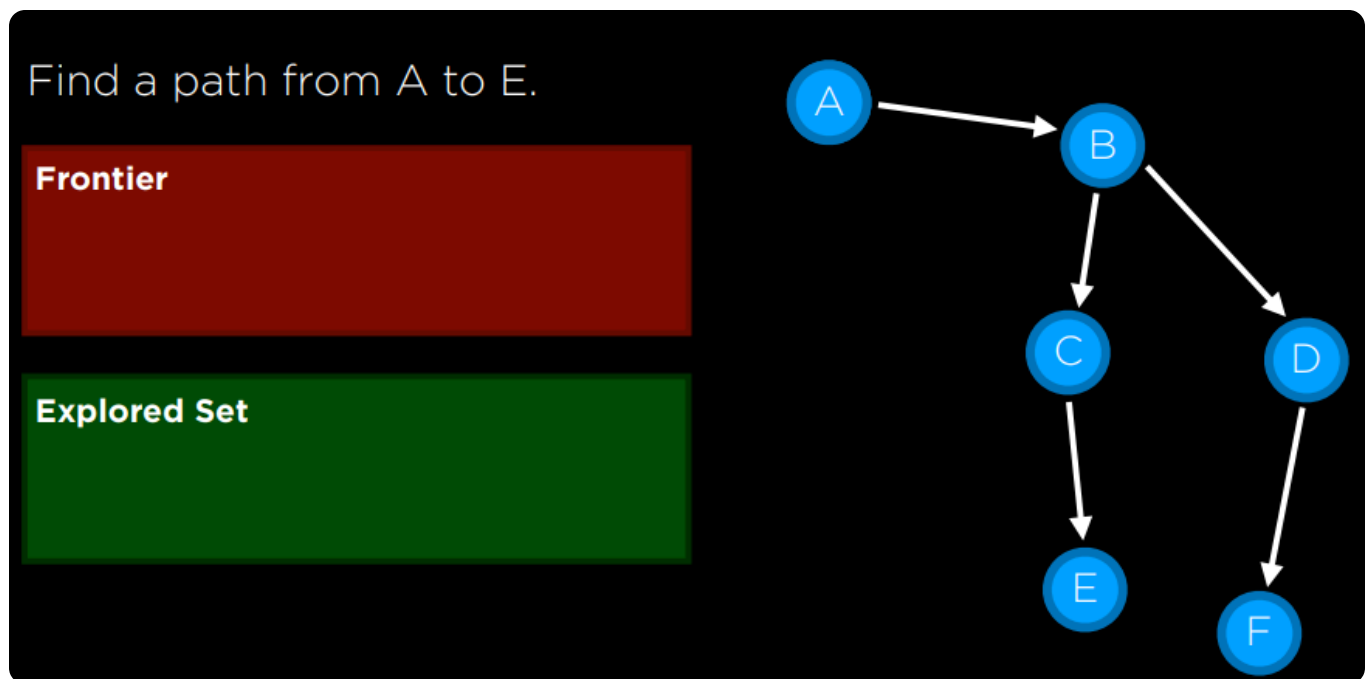
Behavior

- **Exploration Strategy:** BFS explores all nodes at the **current depth** (distance from the start node) before moving to nodes at the next depth level.
 - It systematically checks all nodes that are one step away from the start, then all nodes two steps away, and so on, progressing layer by layer through the search tree or graph.
 - This "breadth-first" approach ensures that the **shallowest nodes** (those closest to the start in terms of path length) are explored first.

- **Key Characteristic:** BFS spreads outward from the start node, exploring all possible paths in a level-by-level manner, which makes it ideal for finding the **shortest path** in unweighted graphs.
- **Intuitive Analogy:** Imagine dropping a stone in a pond and watching the ripples spread outward. BFS explores the graph like these ripples, covering all nodes at a given distance before moving farther out.
- **Backtracking:** Unlike DFS, BFS does not backtrack in the same way. Instead, it exhaustively explores all nodes at the current depth before proceeding, ensuring no deeper nodes are explored prematurely.

Example on a Graph (A → E)

Consider a directed graph with nodes {A, B, C, D, E, F} and edges: A → B, B → C, B → D, D → F, C → E. The goal is to find a path from node A to node E.



1. Initialization:

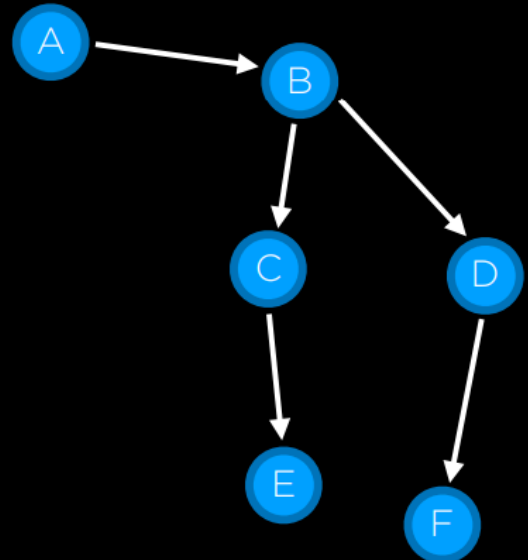
- Frontier = {A} (start node enqueued).
- Explored = {} (set of visited nodes, initially empty).
- Parent = {} (dictionary to track the parent of each node for path reconstruction).

Find a path from A to E.

Frontier



Explored Set



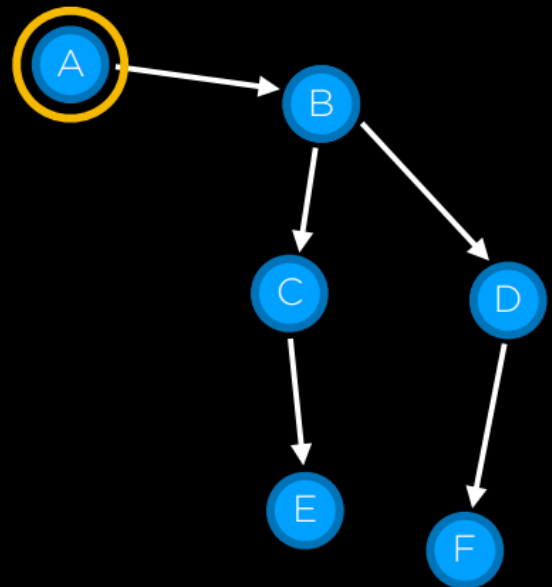
2. **Step 1:**

- Dequeue A from the frontier (Frontier = {}).

Find a path from A to E.

Frontier

Explored Set



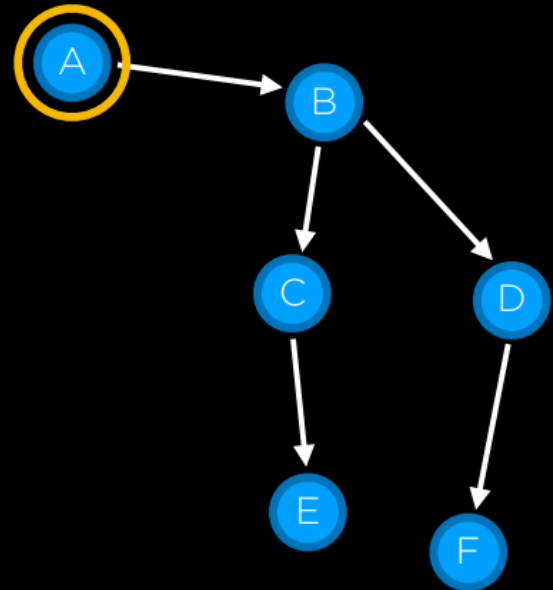
- A is not the goal (E). Add A to Explored = {A}.
- A has one neighbor: B. Enqueue B.
- Frontier = {B}, Explored = {A}, Parent = {B → A}.

Find a path from A to E.

Frontier



Explored Set



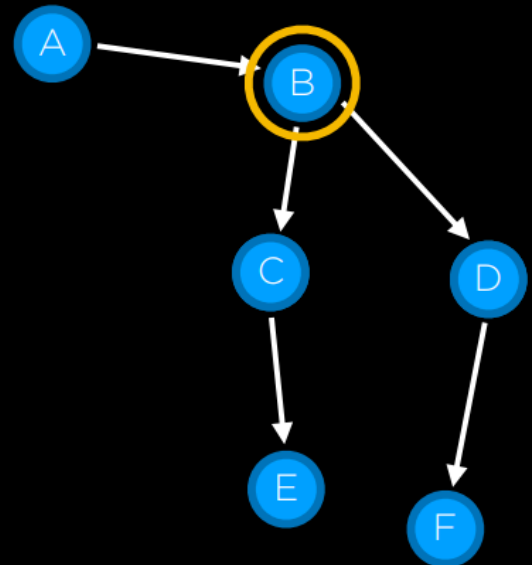
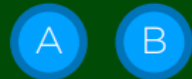
3. **Step 2:**

- Dequeue B from the frontier (Frontier = {}).

Find a path from A to E.

Frontier

Explored Set



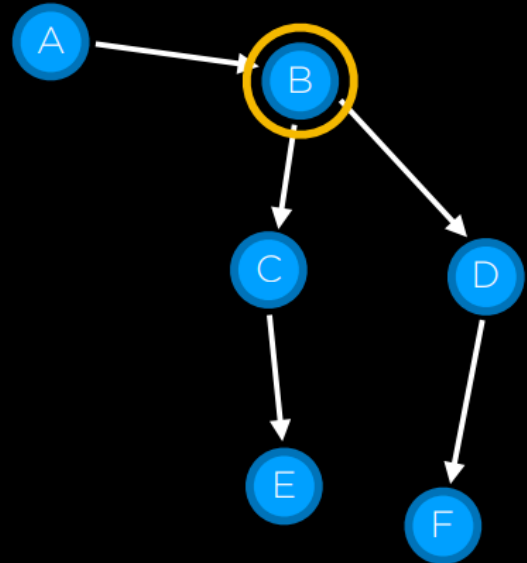
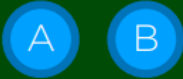
- B is not the goal. Add B to Explored = {A, B}.
- B has two neighbors: C and D. Enqueue them in queue order (C, then D).
- Frontier = {C, D}, Explored = {A, B}, Parent = {B → A, C → B, D → B}.

Find a path from A to E.

Frontier



Explored Set



4. **Step 3:**

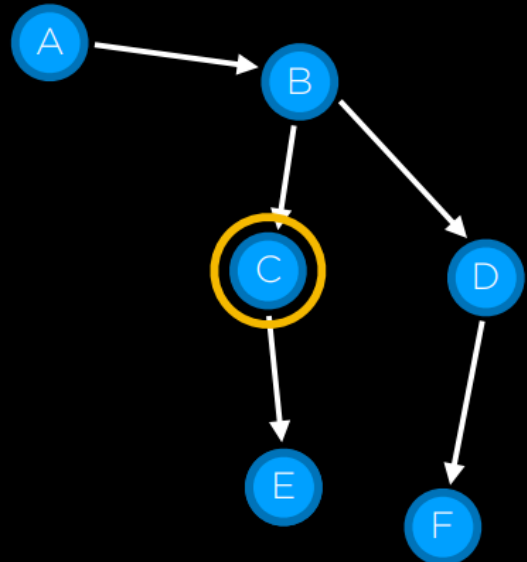
- Dequeue C from the frontier (Frontier = {D}).
- C is not the goal. Add C to Explored = {A, B, C}.

Find a path from A to E.

Frontier



Explored Set



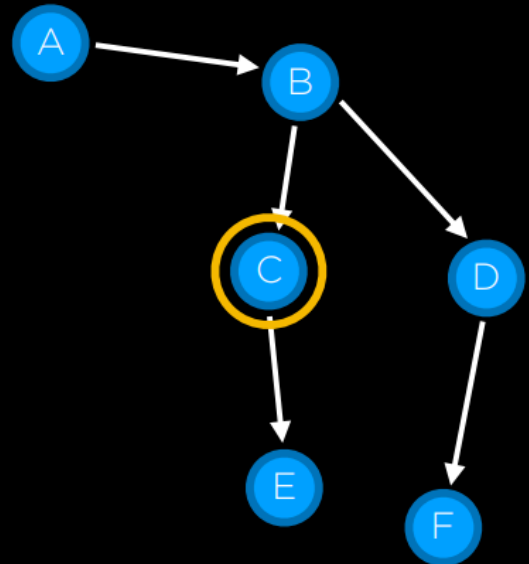
- C has one neighbor: E. Enqueue E.
- Frontier = {D, E}, Explored = {A, B, C}, Parent = {B → A, C → B, D → B, E → C}.

Find a path from A to E.

Frontier



Explored Set



5. **Step 4:**

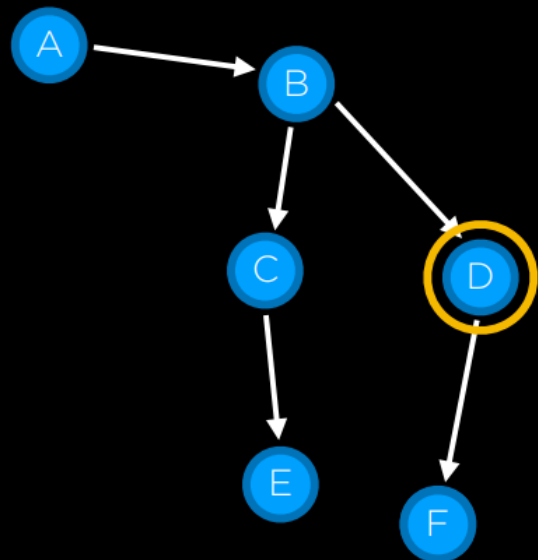
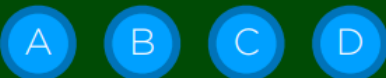
- Dequeue D from the frontier (Frontier = {E}).

Find a path from A to E.

Frontier



Explored Set



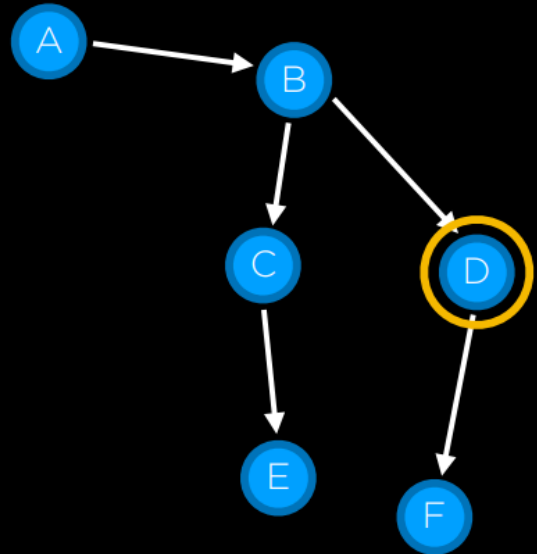
- D is not the goal. Add D to Explored = {A, B, C, D}.
- D has one neighbor: F. Enqueue F.
- Frontier = {E, F}, Explored = {A, B, C, D}, Parent = {B → A, C → B, D → B, E → C, F → D}.

Find a path from A to E.

Frontier



Explored Set



6. **Step 5:**

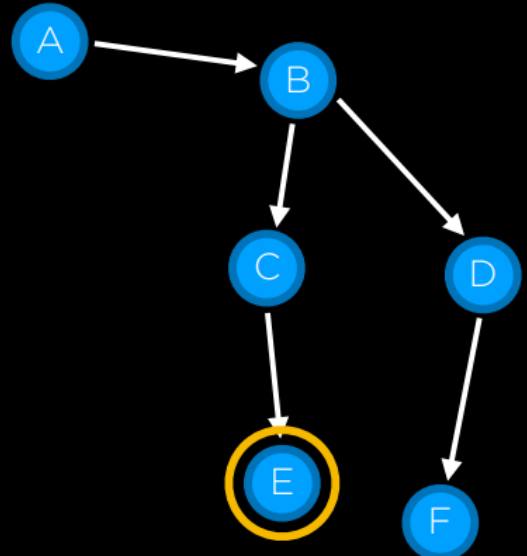
- Dequeue E from the frontier (Frontier = {F}).

Find a path from A to E.

Frontier



Explored Set



- E is the goal! Stop the search.
- Reconstruct the path using the Parent dictionary: $E \rightarrow C \rightarrow B \rightarrow A$ (reverse order: $A \rightarrow B \rightarrow C \rightarrow E$).
- **Solution Path:** $A \rightarrow B \rightarrow C \rightarrow E$.

Note: The path $A \rightarrow B \rightarrow C \rightarrow E$ is guaranteed to be the **shortest path** (in terms of the number of edges) from A to E in an unweighted graph, as BFS explores nodes in order of increasing distance from the start.

Maze Example

- **Scenario:** Imagine navigating a maze where each cell is a state, and edges represent valid moves (e.g., up, down, left, right) to adjacent cells without walls.
- **BFS in a Maze:**
 - Start at the entrance (initial state).
 - Explore all cells that are one step away from the entrance, then all cells two steps away, and so on, moving outward layer by layer.
 - If the exit (goal state) is reachable, BFS will find the **shortest path** to it (in terms of the number of moves).
- **Visualization:**
 - BFS explores the maze like a wave spreading outward, checking all possible moves at the current distance before moving farther.
 - For example, if the entrance is at cell (0,0), BFS first checks all adjacent cells (e.g., (0,1), (1,0)), then all cells two steps away (e.g., (0,2), (1,1), (2,0)), and so on.
 - This ensures that if multiple paths to the exit exist, BFS finds the one with the fewest steps.
- **Example Path:**
 - Suppose the maze has an exit 5 steps away (via the shortest path) but also a longer path of 20 steps. BFS will find the 5-step path because it explores all nodes at depth 5 before considering deeper nodes.
 - In contrast, DFS might explore the 20-step path first and return it, missing the shorter solution.

Properties

1. **Completeness:**

- BFS is **complete** in **finite state spaces**, meaning it will find a solution if one exists.
- In **infinite state spaces** or graphs with cycles, BFS remains complete if cycle detection is implemented (e.g., by maintaining an Explored set to avoid revisiting nodes).
- **Solution:** Use an Explored set to prevent re-exploring nodes in graphs with cycles, ensuring termination in finite graphs.

2. **Optimality:**

- BFS is **optimal** in unweighted graphs or when all actions have equal cost (e.g., each edge has a cost of 1).
- It guarantees the **shortest path** (fewest edges) to the goal because it explores nodes in order of increasing distance from the start node.
- In weighted graphs (where edges have different costs), BFS is not optimal unless modified (e.g., using Dijkstra's algorithm for non-negative edge weights).

3. Memory Efficiency:

- BFS is **less memory-efficient** compared to DFS.
- It stores all nodes at the current depth level in the frontier, which can grow exponentially in wide search trees or graphs with a high branching factor.
- For example, in a binary tree with depth (d), the frontier may store up to (2^d) nodes at the deepest level, leading to high memory usage.
- BFS also maintains:
 - The **Explored set** to track visited nodes and avoid cycles.
 - A **Parent dictionary** to reconstruct the solution path.
- **Trade-off**: While BFS guarantees the shortest path, its memory requirements make it less suitable for large state spaces with high branching factors.

4. Time Complexity:

- In the worst case, BFS explores all nodes and edges in the state space.
- For a graph with (V) nodes and (E) edges, the time complexity is **$O(V + E)$** with an adjacency list representation.
- In a tree with branching factor (b) and depth (d), the worst-case time complexity is **$O(b^d)$** if the entire tree is explored.

5. Space Complexity:

- The space complexity is dominated by the frontier, which can grow to include all nodes at the deepest level explored.
- In a tree, the worst-case space complexity is **$O(b^d)$** , where (b) is the branching factor and (d) is the depth of the shallowest goal.
- In a graph, the space complexity is **$O(V)$** for the Explored set and frontier combined, assuming a high branching factor.

Additional Notes

• Applications:

- BFS is used when finding the **shortest path** or minimum number of steps is critical, such as:
 - Pathfinding in navigation systems (e.g., GPS, maze-solving robots).
 - Social network analysis (e.g., finding the shortest connection between two people).
 - Puzzle-solving (e.g., Rubik's Cube, where the minimum number of moves is desired).
 - Network broadcasting (e.g., spreading a message to all nodes in the fewest steps).

- **Variants:**
 - **Bidirectional BFS:** Runs BFS from both the start and goal nodes simultaneously, meeting in the middle to reduce the search space (especially effective in large graphs).
 - **Uniform-Cost Search (UCS):** A generalization of BFS for weighted graphs, where edges have different costs, ensuring the minimum-cost path is found.
- **Advantages:**
 - Guarantees the shortest path in unweighted graphs.
 - Simple to implement using a queue.
 - Well-suited for problems where the goal is close to the start node in terms of edge count.
- **Disadvantages:**
 - High memory usage, making it impractical for large state spaces with high branching factors.
 - Slower in deep search spaces compared to DFS, as it explores all nodes at each depth.
- **Implementation Tip:**
 - Use a **deque** (double-ended queue) for efficient enqueue and dequeue operations (e.g., `collections.deque` in Python).
 - Maintain an **Explored set** for $O(1)$ lookup to avoid revisiting nodes in graphs with cycles.
 - Use a **Parent dictionary** to track the path for solution reconstruction.

Pseudocode for BFS (Graph Search)

```
function BFS(graph, start, goal):  
    frontier = Queue() # Initialize queue for frontier  
    frontier.enqueue(start)  
    explored = Set() # Track visited nodes  
    parent = Dictionary() # Track path (parent[node] = previous node)  
  
    while frontier is not empty:  
        current = frontier.dequeue() # Get the oldest node  
        if current == goal:  
            return reconstruct_path(parent, goal) # Goal found, return path  
        if current not in explored:  
            explored.add(current) # Mark as visited  
            for neighbor in graph.neighbors(current):  
                if neighbor not in explored and neighbor not in frontier:  
                    frontier.enqueue(neighbor) # Add unvisited neighbors
```

PYTHON

```

        parent[neighbor] = current # Track parent for path

    return None # No solution found

def reconstruct_path(parent, goal):
    path = [goal]
    current = goal
    while current in parent:
        current = parent[current]
        path.append(current)
    return reverse(path) # Return path from start to goal

```

Comparison: DFS vs. BFS

- **Maze Example (Maze2.txt):**
 - **DFS:** Explored 399 states, following long paths and backtracking extensively.
 - **BFS:** Explored only 77 states, finding the same optimal path more efficiently by exploring shallow nodes first.
 - Visualization shows BFS leaves more unexplored cells, focusing on nodes closer to the start.
- **Trade-offs:**
 - DFS is memory-efficient but may explore unnecessary paths.
 - BFS is optimal for shortest paths but requires more memory.

Example: Maze3.txt

In a small maze with multiple paths from A to B:

- **DFS:** May choose a longer path (e.g., 10 steps) if unlucky, despite a shorter path (e.g., 5 steps) existing.
- **BFS:** Finds the shortest path by exploring all nodes at increasing distances from A.

↻

Uninformed Search

- **Definition:** Uninformed search strategies, also known as blind search, explore the search space without any domain-specific knowledge or heuristic guidance about the goal's location or cost. They rely solely on the problem's structure (e.g., nodes, edges, and actions).
- **Characteristics:**
 - Use only the information provided in the problem definition (e.g., start state, actions, transition model).
 - Explore the search space systematically, often in a predefined order (e.g., depth-first or breadth-first).
 - No estimation of how close a node is to the goal.
- **Examples:**
 - **Breadth-First Search (BFS):** Explores all nodes at the current depth before moving to deeper nodes, ensuring the shortest path in unweighted graphs.
 - **Depth-First Search (DFS):** Explores as far as possible along one path before backtracking, memory-efficient but not optimal.
 - **Uniform-Cost Search (UCS):** Expands nodes in order of increasing path cost, optimal for weighted graphs with non-negative costs.
- **Properties:**
 - **Completeness:** Guaranteed to find a solution in finite state spaces (with cycle detection for graphs).
 - **Optimality:** Depends on the algorithm (e.g., BFS and UCS are optimal for unweighted and weighted graphs, respectively; DFS is not).
 - **Drawback:** Can be inefficient, as they explore all possibilities without prioritizing promising paths.
- **Use Case:** Suitable when no additional information about the problem is available, such as in simple mazes or puzzles with no heuristic guidance.

Informed Search

- **Definition:** Informed search strategies, also known as heuristic search, use domain-specific knowledge or heuristics to guide the search toward the goal more efficiently. Heuristics estimate the cost or distance to the goal, prioritizing nodes that appear closer.
- **Characteristics:**
 - Incorporate a **heuristic function** ($h(n)$), which estimates the cost from a node (n) to the goal.
 - Aim to reduce the search space by exploring nodes likely to lead to the goal faster.

- Heuristics must be carefully designed to be **admissible** (never overestimate the true cost) for optimality in some algorithms.
- **Examples:**
 - **Greedy Best-First Search:** Expands nodes with the lowest heuristic value ($h(n)$), prioritizing nodes that seem closest to the goal, but not necessarily optimal.
 - **A* Search:** Combines the cost to reach a node ($g(n)$) with the heuristic estimate ($h(n)$), expanding nodes with the lowest ($f(n) = g(n) + h(n)$). Optimal if the heuristic is admissible and consistent.
 - **Weighted A*:** Modifies **A*** by scaling the heuristic (e.g., ($f(n) = g(n) + w \cdot h(n)$)) for faster search, trading optimality for efficiency.
- **Properties:**
 - **Completeness:** Complete if the heuristic is admissible and the state space is finite.
 - **Optimality:** A* is optimal with admissible and consistent heuristics; others like Greedy Best-First are not.
 - **Advantage:** More efficient than uninformed search in large or complex state spaces, as heuristics guide the search toward the goal.
 - **Drawback:** Requires a good heuristic, which can be challenging to design. Poor heuristics may reduce efficiency or lead to suboptimal solutions.
- **Use Case:** Ideal for problems with large state spaces where heuristics can estimate proximity to the goal, such as pathfinding in navigation systems, game AI, or puzzle-solving (e.g., 8-puzzle).

Key Differences

- **Guidance:** Uninformed search is blind, relying on systematic exploration, while informed search uses heuristics to prioritize promising nodes.
- **Efficiency:** Informed search is typically more efficient in large state spaces, as it focuses on paths likely to lead to the goal.
- **Optimality:** Uninformed search (e.g., BFS, UCS) can be optimal but may explore more nodes; informed search (e.g., A*) is optimal with admissible heuristics and often explores fewer nodes.
- **Complexity:** Informed search requires designing a heuristic, adding complexity, while uninformed search is simpler to implement but less efficient.

Important Definitions U Need

Heuristic

Definition: A heuristic is a problem-solving technique or rule of thumb used to make decisions or guide a search process toward a solution more efficiently, especially when complete information is unavailable or the search space is large.

Purpose in Search: In search algorithms (e.g., A* or Greedy Best-First Search), heuristics provide an estimate of the cost or distance from a given state (node) to the goal, helping prioritize which nodes to explore.

Characteristics:

- Based on domain-specific knowledge or assumptions about the problem.
- Not guaranteed to be accurate but aims to be computationally inexpensive and useful.
- **Example:** In a maze, the **Manhattan distance** (straight-line steps ignoring walls) is a heuristic to estimate the distance to the exit.

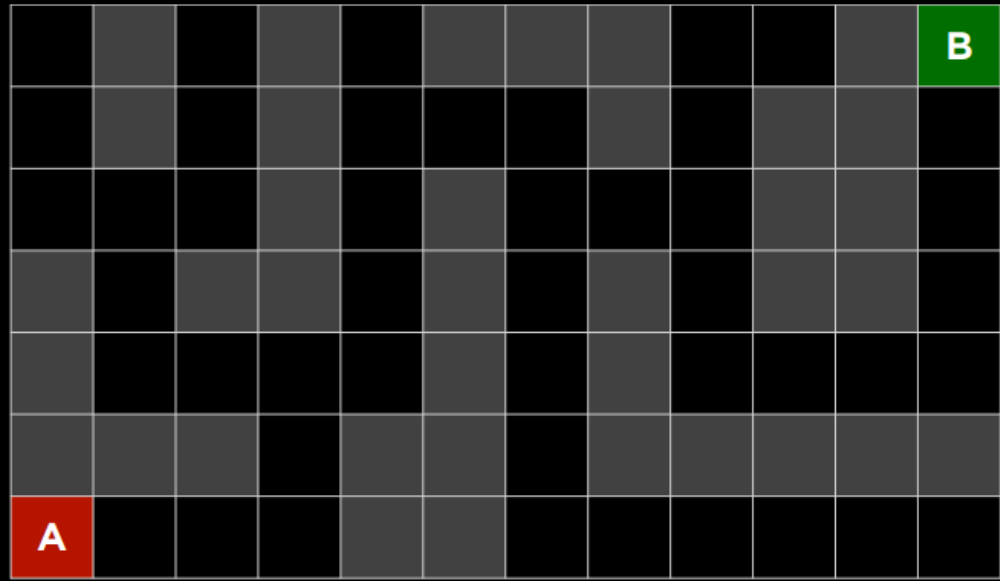
Trade-off: Heuristics improve efficiency by reducing the number of explored nodes but may lead to suboptimal solutions if poorly designed.

∅

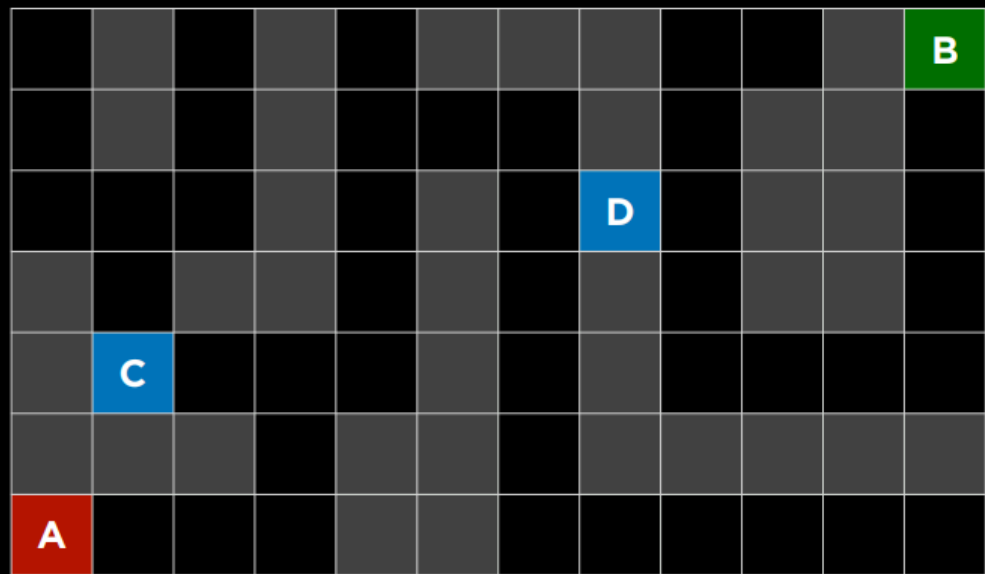
Heuristic Function ($h(n)$)

Definition: The heuristic function, denoted as $h(n)$, is a mathematical function that estimates the cost (e.g., distance, steps, or effort) from a node n to the goal state in a search problem.

Heuristic function?



Heuristic function?



Role in Informed Search:

- Used in algorithms like A* and Greedy Best-First Search to guide the exploration of the search space.
- For a node n , $h(n)$ provides a numerical estimate of the remaining cost to reach the goal, influencing which nodes are prioritized in the frontier.

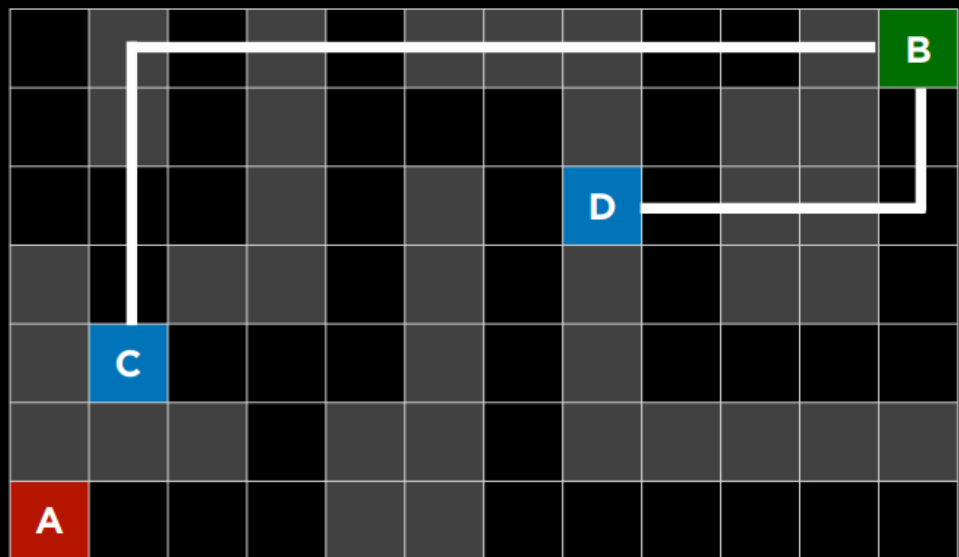
Desirable Properties:

- **Admissible:** $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to the goal. An admissible heuristic never overestimates the cost, ensuring optimality in algorithms like A*.
- **Consistent (Monotonic):** For every node n and successor n' , $h(n) \leq h(n') + c(n, n')$, where $c(n, n')$ is the cost of moving from n to n' . This ensures the heuristic is locally consistent.

Example:

- In a maze, for a node at (x_1, y_1) and goal at (x_2, y_2) , the heuristic function might be the Manhattan distance: $h(n) = |x_2 - x_1| + |y_2 - y_1|$.

Heuristic function? Manhattan distance.



- For the 8-puzzle, $h(n)$ could be the number of misplaced tiles or the sum of Manhattan distances of each tile to its goal position.

Impact: A well-designed $h(n)$ reduces the search space by focusing on promising paths, while a poor heuristic may lead to inefficiency or suboptimal solutions.



Advanced Search Algorithms

Greedy Best-First Search

Idea

- **Core Concept:** Greedy Best-First Search is an informed search algorithm that uses a heuristic function ($h(n)$) to estimate the cost (or distance) from a node (n) to the goal. It always selects the node from the frontier with the lowest $h(n)$, prioritizing paths that appear closest to the goal based on the heuristic.
- **Heuristic-Driven:** Unlike uninformed searches (e.g., BFS, DFS), it leverages domain-specific knowledge encoded in the heuristic to guide the search, aiming to reduce the number of explored nodes.
- **Greedy Nature:** The algorithm is "greedy" because it makes locally optimal choices at each step, choosing the node with the smallest heuristic value without considering the cost to reach that node.

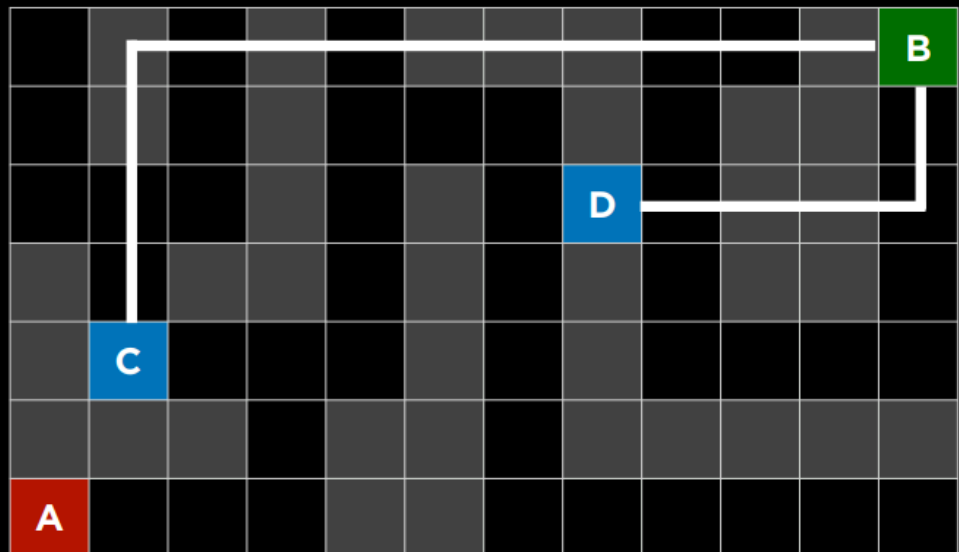
Heuristic Example

- **Manhattan Distance in a Maze:** In a grid-based maze, where movement is restricted to up, down, left, or right, the Manhattan distance is a common heuristic. For a node at position (x_1, y_1) and a goal at (x_2, y_2) , the Manhattan distance is calculated as:

$$h(n) = |x_2 - x_1| + |y_2 - y_1|$$

This estimates the number of steps to the goal, assuming no obstacles (walls), making it an admissible heuristic (it never overestimates the true cost in a maze without diagonal movement).

Heuristic function? Manhattan distance.



- **Other Heuristics:**
 - **Euclidean Distance:** For problems allowing diagonal or free movement, use the straight-line distance: $h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

- **Custom Heuristics:** For puzzles like the 8-puzzle, use the number of misplaced tiles or the sum of Manhattan distances of each tile to its goal position.

Frontier

- **Implementation:** The frontier is implemented as a priority queue, where nodes are prioritized based on their heuristic value ($h(n)$). Nodes with the lowest $h(n)$ are dequeued first, ensuring the algorithm explores paths that seem closest to the goal. Ties (equal $h(n)$) can be broken arbitrarily or using secondary criteria (e.g., FIFO order).
- **Data Structure:** Typically implemented using a min-heap or priority queue library (e.g., `heapq` in Python) for efficient retrieval of the node with the smallest $h(n)$.

Example

- **Maze Scenario:** Consider a grid-based maze where each cell is a node, and the goal is to reach cell B (the goal) from a starting cell. Each cell is labeled with its Manhattan distance to B (ignoring walls for the heuristic).

At a decision point, the algorithm evaluates two possible moves:

- **Left:** $h(n) = 13$ (13 steps estimated to reach B).
- **Right:** $h(n) = 11$ (11 steps estimated to reach B).

The algorithm chooses Right because $h(n) = 11 < 13$.

As it progresses, it continues selecting nodes with lower heuristic values (e.g., $h(n) = 10, 9, 8$).

- **Potential Issue:** If the right path leads to a dead end (e.g., a wall), the algorithm must backtrack by selecting the next node from the priority queue, which could be a previously ignored node (e.g., the left path with $h(n) = 13$).
- **Path Behavior:** The algorithm follows a path that appears promising based on the heuristic but may encounter obstacles or longer routes. For example, a path with decreasing $h(n)$ values (e.g., 11, 10, 9) might lead to a dead end, forcing the algorithm to explore alternative paths with higher $h(n)$.

Properties

- **Efficiency:** Greedy Best-First Search is more efficient than uninformed searches like BFS because it explores fewer nodes by prioritizing those with lower heuristic values. However, its efficiency depends heavily on the quality of the heuristic. A poorly designed heuristic may lead to exploring irrelevant paths.
- **Optimality:** Not guaranteed to find the shortest path, even in unweighted graphs. The algorithm's greedy nature means it may choose a path with low $h(n)$ that is ultimately longer or leads to a dead end, missing shorter paths with initially higher heuristic values.

- **Example Issue:** In the maze, choosing the right path ($h(n) = 11$) over the left ($h(n) = 13$) might lead to a 20-step path due to obstacles, while the left path could have led to a 15-step solution.
- **Completeness:** Complete in finite state spaces if cycle detection is used (e.g., maintaining an Explored set to avoid revisiting nodes). In infinite state spaces or graphs with cycles, it may get stuck without proper cycle detection.
- **Time Complexity:** In the worst case, explores all nodes, giving $O(V + E)$ for a graph with V nodes and E edges, or $O(b^d)$ for a tree with branching factor b and depth d .
- **Space Complexity:** $O(V)$ for graphs (due to the frontier and Explored set) or $O(b^d)$ for trees, depending on the branching factor and depth.

Example Issue

In the maze example, choosing a path with $h(n) = 11$ over $h(n) = 13$ may lead to a dead end or a longer path (e.g., navigating around obstacles). A shorter path via the node with $h(n) = 13$ might exist but be ignored initially due to the greedy selection. This highlights the algorithm's limitation: it prioritizes apparent proximity to the goal over the actual path cost, potentially leading to suboptimal solutions.

»

A* Search

Idea

- **Core Concept:** A* Search is an informed search algorithm that combines the cost to reach a node ($g(n)$, the cumulative cost from the start to node n) with the heuristic estimate to the goal ($h(n)$), prioritizing nodes with the lowest total estimated cost ($f(n) = g(n) + h(n)$).
- **Balanced Approach:** Unlike Greedy Best-First Search, which only considers $h(n)$, A* balances the known cost ($g(n)$) with the estimated cost ($h(n)$) to make more informed decisions, aiming for both efficiency and optimality.
- **Goal:** Find the optimal path (shortest or least-cost) to the goal by exploring nodes that minimize the total estimated cost.

Frontier

- **Implementation:** The frontier is implemented as a priority queue, where nodes are prioritized based on their total estimated cost ($f(n) = g(n) + h(n)$). Nodes with the lowest

$f(n)$ are dequeued first, ensuring the algorithm explores paths that are both close to the start (low $g(n)$) and appear close to the goal (low $h(n)$). Ties can be broken based on secondary criteria (e.g., preferring lower $h(n)$ or FIFO order).

- **Data Structure:** Typically implemented using a min-heap or priority queue library for efficient retrieval of the node with the smallest $f(n)$.

Example

- **Maze Scenario:** Consider the same grid-based maze as in Greedy Best-First Search, with the goal at cell B. Use Manhattan distance as the heuristic ($h(n)$).

At a decision point, evaluate two possible moves:

- **Left:** $g(n) = 6$ (6 steps taken from the start), $h(n) = 13$ (Manhattan distance to B), so $f(n) = 6 + 13 = 19$.
- **Right:** $g(n) = 6$, $h(n) = 11$, so $f(n) = 6 + 11 = 17$.
Choose Right because $f(n) = 17 < 19$.

At a later decision point:

- **Top:** $g(n) = 15$, $h(n) = 6$, so $f(n) = 15 + 6 = 21$.
- **Bottom:** $g(n) = 6$, $h(n) = 13$, so $f(n) = 6 + 13 = 19$.
Choose Bottom because $f(n) = 19 < 21$.

This decision process continues, balancing the cost to reach nodes ($g(n)$) with the estimated cost to the goal ($h(n)$), leading to the optimal path to B.

- **Path Behavior:** A* explores paths that minimize the total estimated cost, avoiding the pitfalls of Greedy Best-First Search (e.g., chasing low $h(n)$ into dead ends). It ensures the shortest path is found if the heuristic is admissible.

Properties

- **Optimality:** A* is optimal if the heuristic is:
 - **Admissible:** $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to the goal (i.e., the heuristic never overestimates).
 - **Consistent (or monotonic):** For every node n and successor n' , $h(n) \leq h(n') + c(n, n')$, where $c(n, n')$ is the cost of the action from n to n' . Consistency ensures the heuristic is locally consistent, preventing the need to revisit nodes. With an admissible and consistent heuristic, A* guarantees the shortest path in unweighted or weighted graphs.
- **Efficiency:** A* is more efficient than BFS and DFS because it uses an informed heuristic to focus on promising paths, exploring fewer nodes. The efficiency depends on the heuristic's quality:
 - A heuristic close to the true cost ($h(n) \approx h^*(n)$) reduces the search space significantly.
 - A poor heuristic (e.g., $h(n) = 0$) reduces A* to Uniform-Cost Search, exploring more

nodes.

- **Completeness:** Complete in finite state spaces with cycle detection (e.g., using an Explored set). In infinite state spaces, A* is complete if a solution exists and the heuristic guides the search effectively.
- **Time Complexity:** In the worst case, $O(V + E)$ for graphs or $O(b^d)$ for trees, but a good heuristic significantly reduces the number of explored nodes.
- **Space Complexity:** $O(V)$ for graphs or $O(b^d)$ for trees, due to the frontier and Explored set. Memory usage can be high in large state spaces.

Challenge:

- **Designing a Good Heuristic:** The heuristic must be admissible and ideally consistent to ensure optimality and efficiency.
 - **Example:** In a maze, Manhattan distance is admissible because it assumes no walls, underestimating the true path length.
 - For the 8-puzzle, the number of misplaced tiles or Manhattan distance of tiles to their goal positions are common admissible heuristics.
 - Poor heuristics can lead to exploring more nodes, reducing efficiency.

↻

Pseudocode for A* Search

```
function A_Star(graph, start, goal, h):  
    frontier = PriorityQueue() # Initialize priority queue for frontier  
    frontier.insert(start, f=0 + h(start)) # f(n) = g(n) + h(n)  
    explored = Set() # Track visited nodes  
    g_score = Dictionary() # Cost to reach each node  
    g_score[start] = 0  
    parent = Dictionary() # Track path  
    parent[start] = None  
  
    while frontier is not empty:  
        current = frontier.extract_min() # Get node with lowest f(n)  
        if current == goal:  
            return reconstruct_path(parent, goal) # Goal found  
        if current not in explored:  
            explored.add(current)  
            for neighbor in graph.neighbors(current):
```

PYTHON

```

        if neighbor not in explored:
            tentative_g = g_score[current] + cost(current, neighbor)
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                f_score = tentative_g + h(neighbor)
                frontier.insert(neighbor, f=f_score)
                parent[neighbor] = current

    return None # No solution found

def reconstruct_path(parent, goal):
    path = [goal]
    current = goal
    while parent[current] is not None:
        current = parent[current]
        path.append(current)
    return reverse(path) # Return path from start to goal

```

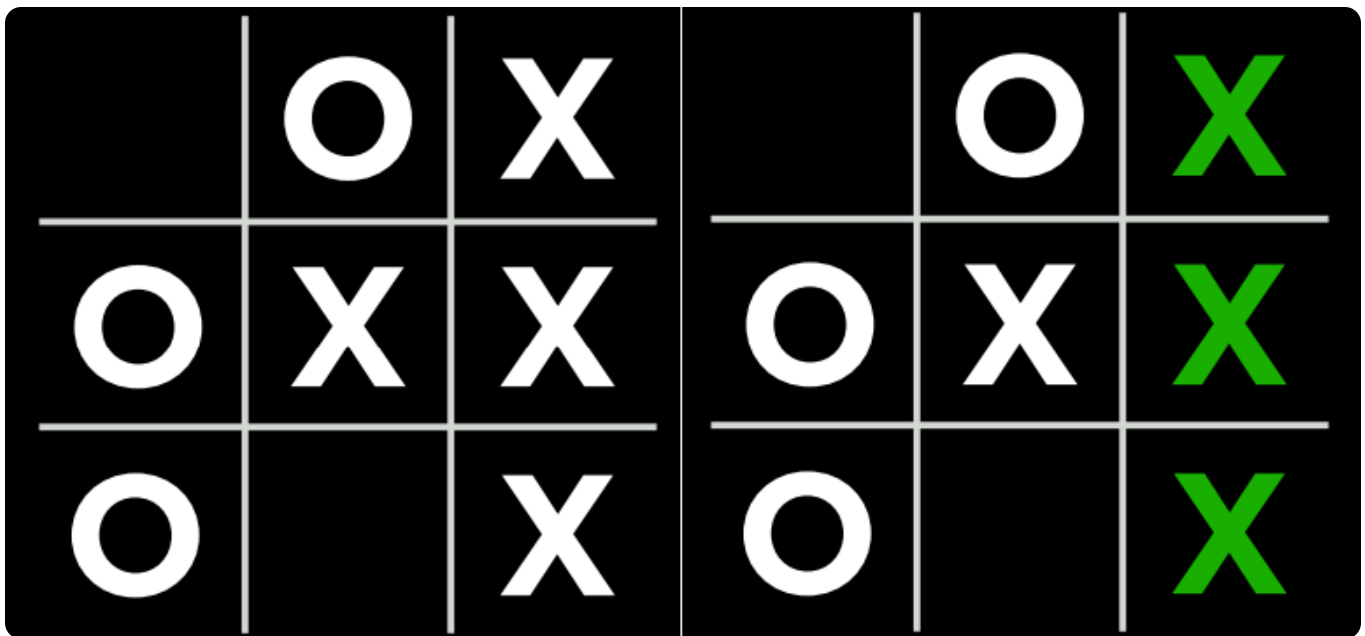


Adversarial Search: Minimax Algorithm



Introduction

Definition: Adversarial search involves decision-making in competitive environments where an agent (e.g., a game-playing AI) competes against an opponent with opposing goals, as seen in two-player, zero-sum games like tic-tac-toe, chess, or checkers. In zero-sum games, one player's gain (positive utility) is the other player's loss (negative utility).



Minimax Algorithm: The **Minimax algorithm** is a decision-making strategy for two-player, zero-sum games with perfect information (both players know the game state and possible moves). It aims to find the optimal move by assuming both players play optimally, maximizing their own utility while minimizing their opponent's.

Key Idea: The algorithm evaluates all possible future game states to determine the best move for the current player, considering the opponent's best responses. It alternates between maximizing the utility for one player (the **max player**) and minimizing it for the opponent (the **min player**).

Applications: Used in games like tic-tac-toe, chess, Go, and other domains involving strategic competition, such as negotiation or military strategy simulations.

Challenges: The algorithm can be computationally expensive for games with large state spaces (e.g., chess), requiring optimizations like alpha-beta pruning or depth-limited search.

☞

Tic-Tac-Toe Example

Setup: Tic-tac-toe is played on a 3x3 grid where two players, X (max player) and O (min player), take turns placing their symbol in an empty cell. The goal is to achieve three symbols in a row (horizontally, vertically, or diagonally).

Outcomes and Utilities:

- **X wins:** Utility = +1 (favorable for X, the max player).
- **O wins:** Utility = -1 (favorable for O, the min player).
- **Draw:** Utility = 0 (neutral outcome).

<table> <tr><td>O</td><td>X</td><td>X</td></tr> <tr><td>O</td><td>O</td><td></td></tr> <tr><td>O</td><td>X</td><td>X</td></tr> </table> -1	O	X	X	O	O		O	X	X	<table> <tr><td>X</td><td>O</td><td>X</td></tr> <tr><td>O</td><td>O</td><td>X</td></tr> <tr><td>X</td><td>X</td><td>O</td></tr> </table> 0	X	O	X	O	O	X	X	X	O	<table> <tr><td>O</td><td></td><td>X</td></tr> <tr><td></td><td>X</td><td>O</td></tr> <tr><td>X</td><td>O</td><td>X</td></tr> </table> 1	O		X		X	O	X	O	X
O	X	X																											
O	O																												
O	X	X																											
X	O	X																											
O	O	X																											
X	X	O																											
O		X																											
	X	O																											
X	O	X																											

Goal:

- **X (max player):** Chooses moves to maximize the utility (aim for +1, settle for 0, avoid -1).
- **O (min player):** Chooses moves to minimize the utility (aim for -1, settle for 0, avoid +1).

- **MAX (X)** aims to maximize score.
- **MIN (O)** aims to minimize score.

Game Dynamics:

- Players alternate turns, with each move generating a new board state.
- The game ends when a player achieves three in a row (win) or the board is full (draw).
- Minimax evaluates all possible game outcomes to select the move that ensures the best possible utility, assuming optimal play by both sides.

Components of Minimax

The Minimax algorithm relies on the following components to model the game and make decisions:

- **Initial State:** The starting configuration of the game (e.g., an empty 3x3 tic-tac-toe board).
- **Player(s):** A function that takes a game state (S) and returns whose turn it is to move (e.g., X or O in tic-tac-toe).
- **Actions(s):** A function that returns the set of legal moves (actions) available in state (S). For example, in tic-tac-toe, this is the set of empty cells on the board.
- **Result(s, a):** A function that returns the new game state resulting from taking action (A) in state (S). For example, placing an X in a specific cell updates the board accordingly.
- **Terminal(s):** A function that checks if state (S) is a terminal state (game over). In tic-tac-toe, this occurs when a player has three in a row or the board is full.
- **Utility(s):** A function that assigns a numerical value to a terminal state (S):
 - +1: Max player (X) wins.
 - -1: Min player (O) wins.
 - 0: Draw.

- S_0 : initial state
- $\text{PLAYER}(s)$: returns which player to move in state s
- $\text{ACTIONS}(s)$: returns legal moves in state s
- $\text{RESULT}(s, a)$: returns state after action a taken in state s
- $\text{TERMINAL}(s)$: checks if state s is a terminal state
- $\text{UTILITY}(s)$: final numerical value for terminal state s

Role in Minimax: These components define the game's structure, allowing the algorithm to simulate all possible moves and outcomes to determine the optimal strategy.

»

Minimax Algorithm

Objective: For each move, the algorithm selects the action that optimizes the utility, assuming the opponent plays optimally to minimize the utility for the max player or maximize it for the min

player.

Core Mechanism:

- The algorithm constructs a **game tree** where nodes represent game states, edges represent actions, and leaves represent terminal states with utility values.
- It recursively evaluates the game tree, alternating between:
 - **Max player (X)**: Chooses the action that maximizes the utility.
 - **Min player (O)**: Chooses the action that minimizes the utility.
- The value of each node is determined by propagating utilities upward from terminal states.

Pseudocode:

```
function Minimax_Decision(state):
    if Player(state) == MAX:
        return argmax_{a in Actions(state)} MinValue(Result(state, a))
    else:
        return argmin_{a in Actions(state)} MaxValue(Result(state, a))

function MaxValue(state):
    if Terminal(state):
        return Utility(state)
    v = -∞
    for each action a in Actions(state):
        v = max(v, MinValue(Result(state, a)))
    return v

function MinValue(state):
    if Terminal(state):
        return Utility(state)
    v = +∞
    for each action a in Actions(state):
        v = min(v, MaxValue(Result(state, a)))
    return v
```

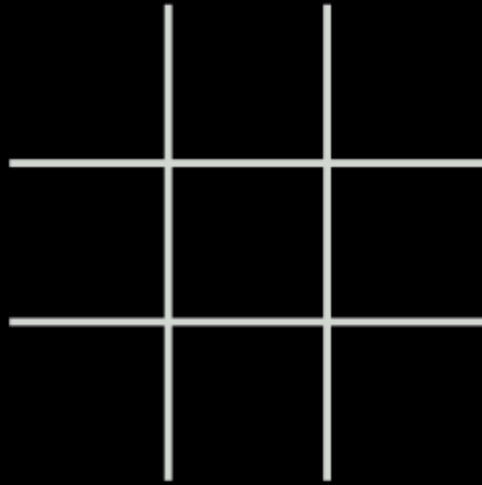
Explanation:

- For the max player, the algorithm evaluates all possible actions and chooses the one leading to the highest utility (via MinValue).
- For the min player, it chooses the action leading to the lowest utility (via MaxValue).
- At terminal states, the utility is returned directly.
- Non-terminal states are evaluated recursively by considering the opponent's optimal response.

Example: Tic-Tac-Toe Decision

Scenario: Consider a tic-tac-toe board where it's O's turn (min player):

Initial State



$\text{PLAYER}(s)$

$\text{PLAYER}\left(\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \right) = \mathbf{X}$

$\text{PLAYER}\left(\begin{array}{|c|c|c|} \hline & & \\ \hline & \mathbf{X} & \\ \hline & & \\ \hline \end{array} \right) = \mathbf{O}$

ACTIONS(s)

$$\text{ACTIONS}\left(\begin{array}{c|c|c} & \text{X} & \text{O} \\ \hline \text{O} & \text{X} & \text{X} \\ \hline \text{X} & & \text{O} \end{array}\right) = \left\{ \begin{array}{c|c|c} \text{O} & & \\ \hline & & \\ \hline & & \end{array}, \begin{array}{c|c|c} & & \\ \hline & & \text{O} \\ \hline & & \end{array} \right\}$$

RESULT(s, a)

$$\text{RESULT}\left(\begin{array}{c|c|c} & \text{X} & \text{O} \\ \hline \text{O} & \text{X} & \text{X} \\ \hline \text{X} & & \text{O} \end{array}, \begin{array}{c|c|c} \text{O} & & \\ \hline & & \\ \hline & & \end{array}\right) = \begin{array}{c|c|c} \text{O} & \text{X} & \text{O} \\ \hline \text{O} & \text{X} & \text{X} \\ \hline \text{X} & & \text{O} \end{array}$$

TERMINAL(s)

$$\text{TERMINAL}\left(\begin{array}{c|c|c} \text{O} & & \\ \hline \text{O} & \text{X} & \\ \hline \text{X} & \text{O} & \text{X} \end{array}\right) = \text{false}$$

$$\text{TERMINAL}\left(\begin{array}{c|c|c} \text{O} & & \text{X} \\ \hline \text{O} & \text{X} & \\ \hline \text{X} & \text{O} & \text{X} \end{array}\right) = \text{true}$$

UTILITY(s)

$$\text{UTILITY}\left(\begin{array}{c|c|c} \text{O} & & \text{X} \\ \hline \text{O} & \text{X} & \\ \hline \text{X} & \text{O} & \text{X} \end{array}\right) = 1$$

$$\text{UTILITY}\left(\begin{array}{c|c|c} \text{O} & \text{X} & \text{X} \\ \hline \text{X} & \text{O} & \\ \hline \text{O} & \text{X} & \text{O} \end{array}\right) = -1$$

Step-by-Step Analysis:

1. **Player(S):** Returns O (min player, as it's O's turn).
2. **Actions(S):** O can place a symbol in empty cells: upper left (1,1) or bottom middle (3,2).

Option 1: O in upper left (1,1):

- New state:

```

x | o |
- - - -
| x |
- - - -
|   | o

```

- Player: X's turn (max player).
- Actions: X can place in bottom middle (3,2).
- Result:

```

x | o |
- - - -
| x |
- - - -
| x | o

```

- Terminal: Game over (board full, no three in a row).
- Utility: Draw, so Utility = 0.

Option 2: O in bottom middle (3,2):

- New state:

```

x |   |
- - - -
| x |
- - - -
| o | o

```

- Player: X's turn (max player).
- Actions: X can place in upper left (1,1).
- Result:

```

x |   | x
- - - -
| x |
- - - -
| o | o

```

- Terminal: Game over (X has three in a row: third column).
- Utility: X wins, so Utility = +1.

Decision:

For O (min player), evaluate the options:

- Upper left (1,1): Utility = 0 (draw).
- Bottom middle (3,2): Utility = +1 (X wins).

O chooses the action that minimizes the utility: upper left (1,1), resulting in a draw (Utility = 0), as it's better than a loss (+1).

Key Insight: Minimax ensures O selects the move that avoids a loss, assuming X plays optimally. The algorithm evaluates the full game tree to make this decision.

↻

Optimization: Alpha-Beta Pruning

Problem: Minimax explores the entire game tree, which is computationally expensive. For example:

- Tic-tac-toe has approximately 255,000 possible games ($9! = 362,880$ possible move sequences, reduced by symmetries and early terminations).
- Chess has an estimated $10^{29,000}$ possible games, making full exploration infeasible.

Solution: Alpha-beta pruning optimizes Minimax by eliminating branches that cannot affect the final decision, significantly reducing the number of nodes evaluated.

Mechanism:

- **Alpha:** The best (highest) utility value found so far for the max player along the current path.
- **Beta:** The best (lowest) utility value found so far for the min player along the current path.
- If a node's value cannot improve alpha (for max) or beta (for min), the algorithm prunes the remaining branches under that node.

Example:

Max player evaluates three actions, leading to min player states with utilities:

- Action 1: Min chooses from {4, 8, 5} → Utility = 4.
- Action 2: Min chooses from {9, 3, ?}.
- Action 3: Min chooses from {2, ?, ?}.

After Action 1, max knows they can achieve at least ($\alpha = 4$).

For Action 2, min evaluates 9 (too high) and 3 (lower than 4). Since min will choose at most 3 ($\beta = 3 < \alpha = 4$), max won't choose this branch, so prune the rest (no need to evaluate ?).

For Action 3, min evaluates 2 ($\beta = 2 < \alpha = 4$), so prune immediately.

Max chooses Action 1 (Utility = 4) without evaluating all nodes.

Efficiency: Alpha-beta pruning can reduce the effective branching factor, potentially exploring only $O(\sqrt[3]{b^d})$ nodes instead of $O(b^d)$, where b is the branching factor and d is the depth.

Requirement: The pruning is most effective with good move ordering (e.g., evaluating likely best moves first).



Depth-Limited Minimax

Problem: For complex games like chess or Go, the game tree is too large to explore fully due to high branching factors (e.g., chess has an average branching factor of 35) and deep trees.

Solution: Depth-limited Minimax restricts the search to a fixed depth (e.g., 10 moves) and uses an **evaluation function** to estimate the utility of non-terminal states at the cutoff depth.

Evaluation Function:

- Estimates the expected utility of a state based on domain-specific features.
- **Example in Chess:**
 - **Material Count:** Assign values to pieces (e.g., pawn = 1, knight = 3, queen = 9) and compute the difference between players (e.g., white's total - black's total).
 - **Positional Factors:** Consider king safety, pawn structure, or control of the center.
 - Example: A state with a score of +0.8 suggests white is likely to win, while -0.8 favors black.
- **Example in Tic-Tac-Toe:** Count potential winning lines for X versus O.

Impact:

- A better evaluation function improves AI performance by providing more accurate estimates of a state's value.
- Poor evaluation functions may lead to suboptimal decisions.

Trade-off: Depth-limited Minimax sacrifices completeness (may miss deeper strategies) for computational feasibility but relies heavily on the evaluation function's quality.

၂၈

Conclusion

Role of Search Algorithms in AI: Search algorithms are foundational to AI, enabling rational decision-making in diverse domains:

- **Classical Search:**
 - **DFS:** Memory-efficient, non-optimal, suitable for deep solutions in large state spaces.
 - **BFS:** Optimal for shortest paths in unweighted graphs, memory-intensive due to large frontier.
 - **Greedy Best-First:** Heuristic-guided, efficient but non-optimal, may miss shorter paths.
 - **A*:** Optimal with admissible and consistent heuristics, balances efficiency and correctness.
- **Adversarial Search:**
 - **Minimax:** Optimal for two-player, zero-sum games, ensures best move assuming optimal opponent play.
 - Optimized with alpha-beta pruning to reduce computation and depth-limited search for complex games.

Applications:

- **Navigation:** Pathfinding in GPS or robotics (e.g., BFS, A* in Google Maps).
- **Puzzle Solving:** Solving games like the 8-puzzle or Rubik's Cube (e.g., A* with heuristics).
- **Game AI:** Strategic decision-making in tic-tac-toe, chess, or Go (e.g., Minimax with alpha-beta pruning).

Next Steps: The exploration of knowledge representation and reasoning follows, focusing on how AIs store, organize, and use information to draw conclusions, make inferences, and solve complex problems beyond search-based approaches.